



Universidade Estadual de Campinas
Instituto de Computação



Jeanfranco David Farfan Escobedo

Context-based intention classification in natural
language dialog flow processing

Classificação de intenções baseada em contexto no
processamento de fluxos de diálogos em linguagem
natural

CAMPINAS
2021

Jeanfranco David Farfan Escobedo

**Context-based intention classification in natural language dialog
flow processing**

**Classificação de intenções baseada em contexto no processamento
de fluxos de diálogos em linguagem natural**

Dissertação apresentada ao Instituto de Computação da Universidade Estadual de Campinas como parte dos requisitos para a obtenção do título de Mestre em Ciência da Computação.

Dissertation presented to the Institute of Computing of the University of Campinas in partial fulfillment of the requirements for the degree of Master in Computer Science.

Supervisor/Orientador: Prof. Dr. Julio Cesar dos Reis

Este exemplar corresponde à versão final da Dissertação defendida por Jeanfranco David Farfan Escobedo e orientada pelo Prof. Dr. Julio Cesar dos Reis.

CAMPINAS
2021

Ficha catalográfica
Universidade Estadual de Campinas
Biblioteca do Instituto de Matemática, Estatística e Computação Científica
Ana Regina Machado - CRB 8/5467

F224c Farfan Escobedo, Jeanfranco David, 1994-
Context-based intention classification in natural language dialogue flow
processing / Jeanfranco David Farfan Escobedo. – Campinas, SP : [s.n.], 2021.

Orientador: Julio Cesar dos Reis.
Dissertação (mestrado) – Universidade Estadual de Campinas, Instituto de
Computação.

1. Processamento de linguagem natural (Computação). 2. Classificação de
intenção. 3. Aprendizado de máquina. 4. Aprendizado por transferência. 5.
Aprendizagem ativa. I. Reis, Julio Cesar dos, 1987-. II. Universidade Estadual
de Campinas. Instituto de Computação. III. Título.

Informações para Biblioteca Digital

Título em outro idioma: Classificação de intenções baseada em contexto no
processamento de fluxos de diálogos em linguagem natural

Palavras-chave em inglês:

Natural language processing (Computer science)

Intent classification

Machine learning

Transfer learning

Active learning

Área de concentração: Ciência da Computação

Titulação: Mestre em Ciência da Computação

Banca examinadora:

Julio Cesar dos Reis [Orientador]

Rodrigo Bonacin

Hélio Pedrini

Data de defesa: 02-03-2021

Programa de Pós-Graduação: Ciência da Computação

Identificação e informações acadêmicas do(a) aluno(a)

- ORCID do autor: <https://orcid.org/0000-0001-6733-6615>

- Currículo Lattes do autor: <http://lattes.cnpq.br/6242935985641657>



Universidade Estadual de Campinas
Instituto de Computação



Jeanfranco David Farfan Escobedo

**Context-based intention classification in natural language dialog
flow processing**

**Classificação de intenções baseada em contexto no processamento
de fluxos de diálogos em linguagem natural**

Banca Examinadora:

- Prof. Dr. Julio Cesar dos Reis
Instituto de Computação - UNICAMP
- Prof. Dr. Hélio Pedrini
Instituto de Computação - UNICAMP
- Dr. Rodrigo Bonacin
CTI Renato Archer

A ata da defesa, assinada pelos membros da Comissão Examinadora, consta no SIGA/Sistema de Fluxo de Dissertação/Tese e na Secretaria do Programa da Unidade.

Campinas, 2 de março de 2021

Acknowledgements

To my eternal heroes, my mother Senovia and father David, because they taught me to work hard for my dreams and never give up. Many thanks!

I am grateful to my family, Werner, Juan Pablo, Delfina, Carlota, Doris and Yuliana, Flor, Zoe, Liam, Marcelo, Aidan, Isharina, for their moral and emotional support. I am also thankful to my other family members for their good wishes. Further, this work is dedicated in perennial memory to my dear cousin Rodrigo, my loved aunt Georgina, my beloved grandparents Basilio and Ricardina, and my other familiars and acquaintances. Those who are in a better life.

Special gratitude to my colleagues from the Laboratory of Information Systems (LIS) in the Institute of Computing (IC). Special mentions to Yusseli, Andre, Gustavo, Marcos, Fabricio, Andressa, and Jacqueline. I will always consider them as my friends. Thanks for all that funny time learning about new things in research and life. A special mention to my Peruvian friends, because they helped me to feel that I am still in my country, especially to Rodolfo, John, and Jose Luis. I have been lucky enough to have them as really close friends.

To my advisor Julio Cesar dos Reis for all his support, I learned a lot during the Master's degree and he was always open to listen to me. Thank you so much! Also, a special mention to my other professors as well: Claudia Medeiros, Ricardo Torres, and Tomasz Kowaltowski, because they showed me new ways to learn and teach.

Finally, I would like to thank Wavy for the financial support. Special mention to Kelly and Fabio, and all the people who work in the Institute of Computing: Denise, Wilson and all others who make our life easier in this wonderful workplace. Thanks to everyone who has had an impact on my life for the past two years, it was an amazing time.

Thank you all!
Jeanfranco David Farfan Escobedo

Resumo

As empresas têm buscado novos canais de conversação para minimizar a interação física entre humanos. Uma alternativa corresponde aos sistemas de conversação, que simulam conversas entre pessoas e máquinas. Esses sistemas possuem um módulo de Entendimento de Linguagem Natural. Este módulo é composto por várias tarefas, que analisam o enunciado do usuário em um quadro semântico, há uma tarefa conhecida como classificação de intenções que visa identificar o que o usuário está tentando realizar a partir de um enunciado. Esta é uma tarefa desafiadora, especialmente se as mensagens forem coletadas de dados reais. Normalmente, as mensagens de texto contêm vários erros gramaticais, outliers, etc. Além disso, a maioria das técnicas de classificação de intenções existentes usa apenas o enunciado atual para prever as intenções de uma determinada consulta. De fato, eles não consideram o papel do contexto (um ou alguns enunciados anteriores) no fluxo do diálogo para essa tarefa. Nesta Dissertação de Mestrado, propomos várias trajetórias baseadas em janelas para investigar o papel da informação contextual para o problema de classificação de intenções, cada trajetória é utilizada para realizar uma concatenação entre a história do diálogo e o enunciado atual. Nosso método de classificação de intenções é baseado em uma rede neural convolucional que obtém representações vetoriais eficazes de BERT para realizar uma classificação precisa de intenções usando qualquer trajetória baseada em janela. Nossos experimentos foram realizados em um corpus de português brasileiro do mundo real com fluxos de diálogo fornecidos pela empresa Wavy global. Finalmente, nossos resultados alcançaram melhorias substanciais em relação à linha de base (sem contexto) em três abordagens usando a expressão do usuário e a resposta do sistema de mensagens anteriores como contexto de diálogo.

Abstract

Companies have sought new conversational channels to minimize human-to-human physical interaction. An alternative corresponds to conversational systems, which simulate conversations between human-to-machine. These systems have a Natural Language Understanding module. This module is composed of several tasks, which parse user utterance into a semantic frame. In this module, there is a task known as intent classification that aims at identifying what a user is attempting to accomplish from an utterance. This is a challenging task especially if messages are collected from real data. Usually, text messages contain several grammatical errors, outliers, *etc.* Also, most existing intent classification techniques use only the current utterance to predict intents of a given query. Indeed, they do not consider the role of the context (one or a few previous utterances) in the dialog flow for this task. In this Master's Dissertation, we propose several windows-based trajectories to investigate the role of contextual information for the intent classification problem. Each trajectory is used to carry out a concatenation between the dialogue history and the current utterance. Our intent classification method is based on a convolutional neural network that obtains effective vector representations from BERT to perform accurate intent classification using a window-based trajectory. Our experiments were carried out on a real-world Brazilian Portuguese corpus with dialog flows provided by Wavy global company. Our results achieved substantial improvements over the baseline (without context) in three approaches using the user utterance and system response from previous messages as dialogue context.

List of Figures

2.1	Subset of a sample word vector space reduced to two dimensions. Given a semantic space, word embeddings with similar meanings tend to appear in the proximity of each other. As highlighted by the word clusters (delimited by the red dashed lines in the figure) associated with big cats, birds and plants. Adapted from Mikolov [55].	22
2.2	Transformer model architecture. Adapted from Vaswani <i>et al.</i> [74].	24
2.3	Scaled Dot-Product Attention (Left). Multi-Head Attention (Right) which consists of several attention layers running in parallel. Adapted from Vaswani <i>et al.</i> [74].	25
2.4	Encoder steps are shown in the left of this figure. Particularities of these steps are shown in more details in the right side of the figure. Adopted from Alammari [1].	26
2.5	Embedding input representation. The input can represent sentence pairs in one token sequence, for instance, <i>my dog is cute</i> (sentence 1) and <i>he likes playing</i> (sentence 2). Sentence pairs are packed together into a single sequence. The first token of every sequence is always a special classification token ([CLS]). Then each sentence is separated with a special token ([SEP]) to delimited the sentence pairs. Adopted from Devlin <i>et al.</i> [18].	28
2.6	A high-level view of BERT model. The input tokens query are “[CLS] my dog is cute [SEP] he likes playing”. Adopted from Chen <i>et al.</i> [10].	28
2.7	Learning process of transfer learning proposed by Tan <i>et al.</i> [73]	29
2.8	Active Learning technique, whose idea is to train and retrain a model in a series of loops. (A) training a classifier with a random selection of samples this has already hand-labeled, (B) predicting labels for the remainder of our data using this model and identify the samples the model was most uncertain about, (C) getting those samples labeled by an oracle, (D) re-training our classifier using the selected random samples, go back to (A) and repeat until there are no more samples without labels.	30
2.9	Confusion Matrix for Binary Classification	31
3.1	Active learning approach with a convolutional neural network architecture with softmax for classification. The preprocessed message feeds the vector representation module that outputs 768-dimension feature vectors (BERT). One-dimension convolution and pooling are applied (Conv1, Conv2/Pooling, and Pooling/Conv3/Pooling). The result is flattened and concatenated in a feature vector (FC4) until the final decision-making stage. Then, the active learner module selects a sample from the remaining training data and carries out active learning algorithms such as uncertainty [6] or ranked batch [51]. Inspired from Zhang [79].	37

3.2	Supervised learning approach using a convolutional neural network architecture with softmax for classification. The preprocessed message feeds the vector representation module that outputs 768-dimension feature vectors (BERT). For this vector, one-dimension convolution and pooling are applied (Conv1, Conv2/Pooling, and Pooling/Conv3/Pooling), and the result is flattened and concatenated in a feature vector (FC4) until the final decision-making stage.	38
3.3	Uncertainty strategy employed with two different stages (from left to right): (a) we used an initial data to train a first model, then we pick up a sample from the remaining data after we compute the uncertainty; (b) if the uncertainty is better than a threshold, we create a new model (second model). We repeat this process until no more samples are available in our training set (remaining data).	41
3.4	Ranked batch strategy employed with two different stages (from left to right): (a) we used a portion initial of data to train a first model. We then compute the uncertainty for the whole remaining data, and pick up the Q lower values to train a new model. The remaining of data is given up to the data training. (b) we update the model and the previous process is repeated until does not have more data.	41
3.5	Accuracy results using uncertainty method on WvGD dataset.	45
3.6	Accuracy results using ranked batch method on WvGD dataset.	45
3.7	Accuracy results using uncertainty method on ATIS dataset.	46
3.8	Accuracy results using ranked batch method on ATIS dataset.	46
4.1	Several types of trajectories (from left to right) in our window-based approach: (a) shows an initial dialog flow; (b) exhibits an approach called ' <i>all context</i> ' that uses the complete previous information (user and bot) as context; Similarly, (c) presents a window-based approach called ' <i>user context</i> ' that employs the user's previous messages to get the context; (d) reveals a window-based approach denominated ' <i>user-bot context</i> ' that employs the last utterance from user and from the bot to get the context; (e) reveals a window-based approach denominated ' <i>last-user context</i> ' that employs the user's last utterance to get the context. Finally, (f) exposes a window-based approach denominated ' <i>last-bot context</i> ' that employs the bot's last utterance to get the context.	50

4.2	Several types of trajectories (from left to right) in our window-based approach: (a) shows a communication process between a user (left) and a bot (right) representing a complete dialog flow, where I_i (blue message) corresponds to i th utterance from the user, and R_j (green message) corresponds to a j th response from a bot. We visualize the current utterance without associated intent as a question mark '?' (red message); (b) exhibits an approach called ' <i>all context</i> ' that uses the complete previous information (user and bot) as context; Similarly, (c) presents a window-based approach called ' <i>user context</i> ' that employs the user's previous messages to get the context; (d) reveals a window-based approach denominated ' <i>user-bot context</i> ' that employs the last utterance from user and from the bot to get the context; (e) reveals a window-based approach denominated ' <i>last-user context</i> ' that employs the user's last utterance to get the context. Finally, (f) exposes a window-based approach denominated ' <i>last-bot context</i> ' that employs the bot's last utterance to get the context.	52
4.3	Distribution of the number of conversations for each dialogue flow size. . .	55
4.4	Distribution of number of intent. At the top of each bar we can see the exact amount of intents. Where 0, 1, ... , 21 correspond to the class labels, for example: initial_greetings , [CT]bank_slip or [IT]delivery_problem. For confidentiality reasons, we cannot reveal more information about the labels	56
4.5	Confusion matrix regarding the results of intent classification without context on WvGD test dataset.	60
4.6	Confusion matrix regarding the results of intent classification using all context on WvGD test dataset.	61
4.7	Confusion matrix regarding the results of intent classification using user context on WvGD test dataset.	61
4.8	Confusion matrix regarding the results of intent classification using the last user-bot context on WvGD test dataset.	62
4.9	Confusion matrix regarding the results of intent classification using the last-user context on WvGD test set	62
4.10	Confusion matrix regarding the results of intent classification using the last-bot context on WvGD test set.	63
4.11	Confusion matrix regarding the results of intent classification with new loss function without context on WvGD test dataset.	65
4.12	Confusion matrix regarding the results of intent classification with new loss function using all context on WvGD test dataset.	65
4.13	Confusion matrix regarding the results of intent classification with new loss function using user context on WvGD test dataset.	66
4.14	Confusion matrix regarding the results of intent classification with new loss function using the last user-bot context on WvGD test dataset.	66
4.15	Confusion matrix regarding the results of intent classification with new loss function using the last-user context on WvGD test dataset.	67
4.16	Confusion matrix regarding the results of intent classification with new loss function using the last-bot context on WvGD test dataset.	67

List of Tables

1.1	An example of user query and the correspondent intent identification. Adapted from Qian <i>et al.</i> [10].	18
1.2	An example of a conversation between user and system. Adapted from Deriu <i>et al.</i> [17].	19
2.1	Comparative analysis regarding existing literature and our proposal.	34
3.1	Convolutional Neural Network hyper-parameters.	40
3.2	Statistical information on ATIS dataset [31].	42
3.3	Occurrences of messages and number of classes in WvGD dataset.	42
3.4	Parameters for each classifier. We highlight in red color the best parameter after a grid search process.	43
3.5	Results with supervised learning approach: classification results obtained by using different vector representations and classifiers.	44
3.6	Supervised learning approach: classification results obtained by using BERT representations and classifiers.	44
4.1	Convolutional Neural Network configuration.	53
4.2	Dataset column detail.	54
4.3	Corpus statistics for the new WvGD dataset.	55
4.4	Comparison between isolated intent classification (without context) and intent classification based on window-based approaches for the WvGD validation set . The highest results are highlighted in red. Baseline results are highlighted in blue.	57
4.5	Comparison between isolated intent classification (without context) and intent classification based on window-based approaches for the WvGD test set . The highest results are highlighted in red. Baseline results are highlighted in blue.	57
4.6	Importance of context during intent classification task. We show two parts, the left part, an utterance without context (red letters), and the right part, an utterance with context (blue letters). In both parts, the query is the same, '2345789' (green letters). However, the intent predicted is different. These results happen because there is a previous dialogue.	58
4.7	Queries in different sessions with a certain level of semantic similarity. Part of the query ' quero mudar equipe' (green), the utterance ' consigo passar novo pedido ' (red) and the message ' novo modelo de negócio ' (orange) are quite similar semantically to the sentence of ' outros assuntos ' (blue). To summarize, these examples represent statement very similars.	59

4.8	Loss function distribution for each intent class. The first column indicates the categories. The second column counts the occurrences of frequent intents. The third and fourth columns show each intent's percentage and loss function computed.	63
4.9	Comparison between isolated intent (without context) and windows-based approaches for intent classification in the WvGD validation dataset using loss values. The highest results are highlighted in red. baseline results are highlighted in blue.	64
4.10	Comparison between isolated intent (without context) and windows-based approaches for intent classification in the WvGD test set using loss values. The highest results are highlighted in red. Baseline results are highlighted in blue.	64

List of Abbreviations

ATIS	Airline Travel Information Systems
BERT	Bidirectional Encoder Representations from Transformer
BERT-DST	BERT Dialogue State Tracking
Bi-LSTM CRF	Bidirectional Long Short-Term Memory with Conditional Random Fields
CAGR	Compound Annual Growth Rate
CNN	Convolutional Neural Networks
CNN-BLSTM	Convolutional Neural Networks and Bidirectional Long Short-Term Memory
	Net-works
CRF	Conditional Random Fields
DCR-Net	Deep Co-Interactive Relation Network
DTS	Dialog State Tracking
FE	Feature Extraction
FT	Fine-Tuning
GOLOMB	GOaL-Oriented Multi-task BERT-based dialogue state tracker
GRUs	Gated Recurrent Units
IC	Institute of Computing
LIS	Laboratory of Information Systems
LR	Logistic Regression
LSTM	Long Short-Term Memory
NER	Named-entity recognition
NLP	Natural Language Processing
NLU	Natural Language Understanding
NN	Neural Network
Q&A	Question and Answer
RELU	Rectified Linear Activation Function
RF	Random Forest
RNN	Recurrent Neural Networks
SOTA	State of the art
SVM	Support Vector Machines
TOD-BERT	Task-Oriented Dialogue BERT
ULMFiT	Universal Language Model
UNICAMP	University of Campinas
WVDS	WaVy Global DataSet

Contents

1	Introduction	16
1.1	Context and Motivation	16
1.2	Problem Characterization	17
1.3	Objectives	19
1.4	Organization of Dissertation	20
2	Background	21
2.1	Fundamental Concepts and Methods	21
2.1.1	Intent Classification	21
2.1.2	Embeddings	22
2.1.3	Transformers	23
2.1.4	Transfer Learning	28
2.1.5	Active Learning	29
2.1.6	Metrics	30
2.2	Related Work	31
3	Investigating Active Learning for Intent Classification on Isolated Utterance	35
3.1	Introduction	35
3.2	Methodology	36
3.2.1	Pre-processing	37
3.2.2	Compute Vectorial Representation	38
3.2.3	Classification	39
3.2.4	Active Learning Module	39
3.3	Experiments	41
3.3.1	Datasets	41
3.3.2	Experimental Settings	42
3.3.3	Computational Resources	43
3.3.4	Results	43
3.4	Discussion	46
3.5	Conclusion	47
4	Intent Classification Relying on Dialog Flow Contextual Information	48
4.1	Introduction	48
4.2	Methodology	49
4.2.1	Dialogue Context Module	49
4.2.2	Feature Extraction Module	51
4.2.3	Classification Module	51
4.3	Experiments	53

4.3.1	Dataset	53
4.3.2	Experimental Settings	55
4.4	Results and Discussion	57
4.5	Conclusion	64
5	Conclusion	68
5.1	Summary of the Contributions	69
5.2	Future Perspectives	70
5.3	Dissemination of Results	70
	Bibliography	71

Chapter 1

Introduction

This chapter describes the context and motivation under investigation in this thesis, as well as its challenges, objectives, contributions, research questions, and text organization.

1.1 Context and Motivation

Nowadays, customers tend to look for quick and accurate solutions. Likewise, customers' needs evolved over the last years. Before, low price and high quality were sought. In these days, a new factor in choosing between competing brands is an additional value of customer service. Customer service is so relevant that 90% of Americans use customer service as a factor in deciding whether or not to do business with a company [48]. Similarly, as stated by [15] consumers are willing to spend 17% more on a company that has outstanding customer service. According to Gallo *et al.* [23], recent surveys have shown that investing in new customers is between 5 and 25 times more expensive than retaining existing ones. As a consequence, a company cannot afford to lose customers.

In this context, the way people communicate with businesses has been evolving at a rapid pace. The rise of real-time messaging has led to a fundamental shift in how people prefer to connect with businesses. According to Chopra [13], 53% of customers are more likely to shop with businesses that they can message. In this context, communication can be made from human-to-human or human-to-machine (dialogue systems or conversational agents), where each utterance in a dialogue is a kind of action being performed by the speaker. These actions are commonly called speech acts, which relies on the Speech Act Theory (SAT) [65]. This theory presented that certain statements are not meant to describe anything, but rather to perform actions.

In a website with thousands of visits a day, live chat employing human-to-human conversation can result in an overwhelming number of “live” conversations to handle. This limitation is only heightened by the use of dialogue systems. Under these circumstances, according to Debecker [16], over 50% of customers expect a business to be available 24/7. Besides, in a medical context, conversational systems have played a leading role by embodying the function of a virtual assistant and bridging the gap between patients and

clinicians [20, 42, 54, 76].

Intent classification can be used against criminal acts such as hate discourses, threats, and criminal slang expression. For instance, Resende *et al.* [61] presented a framework for selecting and classifying social network posts, where ML techniques were used for automatically classifying the posts according to an intention classification.

Conversational agents allow communicate with users in natural language (text, speech, or both). Nevertheless, text communication lacks elements that are natural in face-to-face communication for detecting intentions. Dos Reis *et al.* [19] proposed a set of emoticons designed (and/or chosen) to communicate intentions as an interactive tool to support users in formulating intentions. They are applied to a variety of tasks, for instance, virtual assistants, information-seeking systems and others [17], and fall into two classes: task-oriented dialogue agent and chatbots.

Conventional task-oriented dialogues use conversation with users to help completing tasks and employ a traditional pipeline: (1) Natural Language Understanding (NLU), which parses user utterance into a semantic frame; (2) dialogue state tracking (DST)¹, which updates slots² and values from semantic frames to the latest values for knowledge base retrieval; (3) dialogue policy, which determines an appropriate dialogue act³ for the next system response; and (4) response generation which generates a natural language sequence conditioned on the dialogue act [46].

Chatbots are conversational agents designed to mimic the appearance of informal human conversation [38]. There are two types of chatbots, including close-domain and open-domain chatbots. Unlike closed-domain chatbots, which respond to keywords or intents to accomplish specific tasks, open-domain chatbots can grip in conversation on any topic. According to Kantarci [40], the global chatbot market is expected to reach USD\$1.3 billion by 2025, growing at a CAGR of 24%. This implies that conversational agents have duly increased their importance in the current world scenario.

Within task-oriented dialogue agents and chatbots, one module parses user’s utterances into a semantic frame, which aims at identifying user intent. Formally, intent classification focuses on predicting the intent of a query (natural language sentence). Table 1.1 shows an example of intent classification.

1.2 Problem Characterization

There are two agents that widely interact during intent classification task: *user* and *system*. A conversation is usually structured in turns; each turn is defined by one or more

¹According to Jurafsky [38] dialogue state tracker maintains the current state of the dialogue (which include the user’s most recent dialogue act, plus the entire set of slot-filler constraints the user has expressed so far).

²Whereas slot filling extracts concepts from the sentences, which maybe either persons, organizations, events, etc.

³According to Raheja [60] a dialogue act represents the meaning of an utterance, for instance, each utterance has a corresponding label such as Question or Backchannel (Statement, Non Verbal, or Other).

Table 1.1: An example of user query and the correspondent intent identification. Adapted from Qian *et al.* [10].

Query	I have a meeting at noon and a half sharp, let me know.
Intent	Set an Alarm

utterances from the user and a single system response. As a result of this process of interaction between system and user, the set of turns forms a dialog flow. Usually, when an human-to-human conversation takes place, people “deduce” intention of a sentence based on context of the conversation (one or a few previous utterances). In this sense, people do not interpret an intention based only on an isolated utterance and disconnected intentions within a dialogue flow.

When a dialogue system aims to parse utterance to identify user’s intent, several challenges appear in a dialog flow related to the need of correctly predicting intents from an utterance: (i) an excessive number of turns within a dialog flow makes it difficult to identify the context of the conversation; (ii) it is necessary to review previous intentions to identify what relationship exists with a new sentence content to eventually get the context of the conversation and recognize the most appropriate intention; (iii) within the dialog flow, there is a need to identify the most relevant that help to identify the context of the conversation, and then obtain an accurate recognition.

Table 1.2 shows an example of a conversation between a user and system, in which U represents the user and S system. The Utterance column represents the exchange of dialogues between conversational agents. This column shows three user utterance (red highlights), extremely difficult to classify without a previous context, and the conversational flow column represents the intent for each utterance U .

Under these circumstances, different questions appear. For example: Is the whole dialog flow relevant? Are *greetings* and *goodbye* intent important to get the context? What is the adequate dialog flow window to process the context based on previous intents? Consequently, we understand that it is necessary to manage the dialog flow context to obtain adequate intent detections.

Relevant issues appear when examining the complexity of intent classification task. Shridhar *et al.* [67] indicated that there are countless different intents that a text can have, which makes labeling very difficult. For example, political intents may have labels such as *leftist* or *right-wing*, whereas questions may have intents such as *where to* or *how to*. Also, utterances can contain several grammatical errors, outliers, and many categories. These aspects turn very complex the tasks of intention recognition in chatbot dialog flows.

Table 1.2: An example of a conversation between user and system. Adapted from Deriu *et al.* [17].

Utterance	Conversational flow
U: Hi, I am looking for somewhere to eat	greetings
S: You are looking for a restaurant. What type of food?	
U: I'd like Peruvian food	food_information
S: "La Cusqueñita" is a nice Peruvian restaurant is located near the historic downtown of Cusco.	
U: But, I am allergic to shrimps	user_information
S: Don't worry, "La Cusqueñita" has beef, chicken and fish food	
U: Is it reasonably priced?	cost_information
S: Yes, "La Cusqueñita" is in the moderate price range.	
U: What is the phone number?	request_information
S: The number of "La Cusqueñita" is 385456.	
U: I need to travel the next day to Machu Picchu, is there a tour agency nearby?	another_information
S: Two minutes from "La Cusqueñita" restaurant is the "Inca Travel" tourism agency	
U: Ok, thank you goodbye.	goodbye

1.3 Objectives

This M.Sc. dissertation studies the role of contextual information in the intent classification problem. For this purpose, we use deep learning concepts and previous context information to develop a method capable of achieving competitive results in intent classification tasks. We investigate and compare various n -grams and Deep Learning embeddings in model training to intent classification task on isolated utterances.

We consider these concepts due to the outstanding results achieved in the Dialogue State Tracking tasks [8, 26, 77]. More specifically, we propose the use of a pre-trained architecture based on transformers [74] to guide the process of feature extraction that represents an utterance and context information of a dialog flow to increase the contextual knowledge.

After identifying the best embeddings, we address context to guide the process of learning features that identify user's intent in dialog flow. In particular, we assume that deep learning architectures for dialog flow as context can enhance intent classification task. Our comparative study considers the most popular deep learning architectures for natural language processing.

Our results are derived from the following specific goals:

- Study and compare several embeddings based on n -grams and state-of-the-art deep learning for word representation in Brazilian Portuguese language for isolated utterances;
- Comparative study of classifiers for the intent classification task on isolated utterances.

- Investigate the effects of considering the context in dialog flow for intent classification task in a Brazilian Portuguese dataset.

1.4 Organization of Dissertation

This dissertation is organized as follows: Chapter 2 discusses relevant concepts and existing studies related to our proposal. Chapter 3 presents our study and conducted experiments applying several embedding and classifiers for intent recognition in isolated utterances. Chapter 4 presents our proposed approach based on context information and isolated utterances strategy. Chapter 5 presents the conclusion remarks and summarizes directions for future work.

Chapter 2

Background

This chapter presents relevant concepts related to our research, which includes different techniques for solving intent classification; techniques for utterance representations (embeddings); Transfer Learning; and Active Learning. It also presents a review of the most relevant studies from literature correlated to our proposal.

2.1 Fundamental Concepts and Methods

This section examines important topics to help readers understand our research work, such as intent classification, embeddings, BERT, Transfer Learning and Active Learning.

2.1.1 Intent Classification

Intent Classification [10, 45, 52, 53, 63, 67, 70] is defined as the task of identifying which of a fixed set of actions the user wishes to take based on a given sentence. This definition is aligned with the following question: *what is the user trying to accomplish?*

The origins of intent classification task are related to look over isolated utterances. Schuurmans [63] defines several elements in this context: an utterance $d \in X$ of a dialogue, where X is the utterance space; a fixed set of predefined intents $C = \{c_1, \dots, c_J\}$; a training set D of labeled dialogue utterances $\{d_i, c_i\}_{i=1}^N$, where $(d, c) \in X \times C$. Each d corresponds to one element of C . For instance, $(d, c) = ("I have a meeting at noon and a half sharp, let me know", "Set an Alarm")$.

The evolution of intent classification task is associated with the use of all utterance information. From its early years and up to 2017, the problem considered isolated utterance as input to predict the intent. However, input sentences usually can contain more details for instance: slot filling, sentiment analysis, and language identification [22] to add context information.

Due to the task complexity, the most popular intent isolated classification dataset [31] has been modified in a new version [22]. Consequently, this variant dataset has proposed interesting details for the intent classification task: (i) a scenario in which can

be easily extended to a joint intent classification and slot filling model [10]; (ii) translate an utterance to a more studied language, for instance, from Japanese to English [22]; (iii) definition of queries that are out-of-scope, in other words, queries that do not fall into any of the system’s supported intents [45]; and (iv) multimodal data like Instagram posts requiring modeling a complex relationship between text and image [43].

In our study, we consider the following aspects:

- Address intent classification using only text. Utterances between a *user* and *system* consisting of texts in natural language format;
- Associated only one label to input utterances (single label classification);
- Input utterances were built using a large chat messages corpus in Brazilian Portuguese language.

2.1.2 Embeddings

Vectors for representing words are called embeddings. The proposal is building fixed-length, dense and distributed vector representations for creating a multidimensional semantic space. The goal of embeddings is that words sharing similar context should be close in the vector space (cf. Figure 2.1). The word “embedding” derives from its mathematical sense as a mapping from one space or structure to another [38].

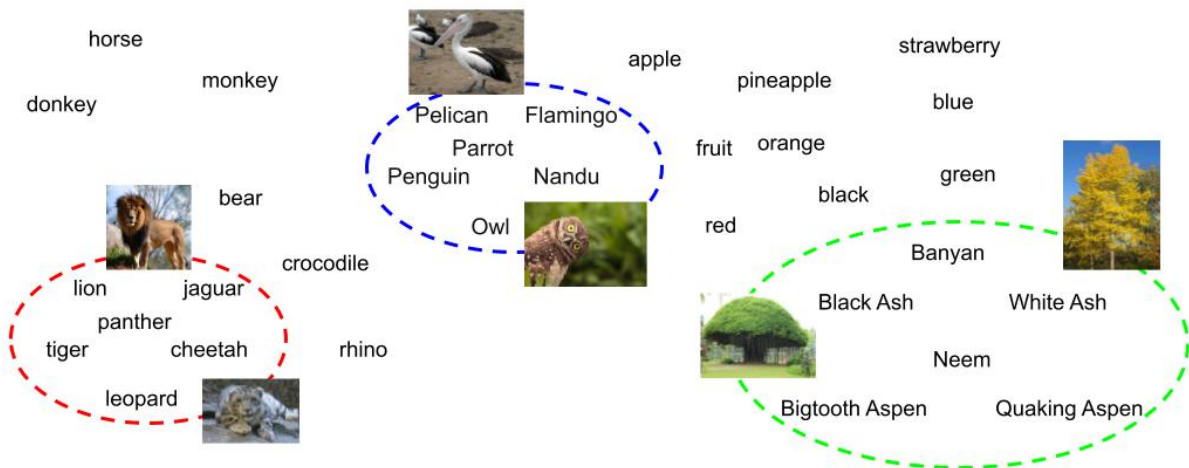


Figure 2.1: Subset of a sample word vector space reduced to two dimensions. Given a semantic space, word embeddings with similar meanings tend to appear in the proximity of each other. As highlighted by the word clusters (delimited by the red dashed lines in the figure) associated with big cats, birds and plants. Adapted from Mikolov [55].

In this investigation, we considered four types of embeddings, as follows: Word2vec [55], Glove [58], FastText [3], and BERT [18] (cf. Section 2.1.3.4). We used this embeddings because the three first embeddings are the most used in Portuguese Language corpus [28] whereas the fourth embedding is based on Transformers (cf. Section 2.1.3) and is considered the state of the art in NLP tasks [74].

- **Word2vec.** This embedding was proposed by Mikolov *et al.* [55]. It uses Neural Networks to learn word embeddings. This can be composed of two different models: Continuous Bag of Words model (CBOW) and Skip-Gram. CBOW model addresses learning word embeddings by training a model to predict a word given its context. Skip-Gram Model is the opposite, addresses learning word embeddings by training a model to predict context given a word.
- **Glove.** This embedding was proposed by Pennington *et al.* [58]. This method consists in a co-occurrence matrix that is constructed by looking at context word-word; each element in the matrix represents the probability of a word close to another word. The model produces a word vector space.
- **FastText.** According to Bojanowski *et al.* [3], FastText is a neural language model trained with large online unlabelled corpora. FastText proposes to learn representations for character n-grams¹, and to represent words as the sum of the n-gram vectors.

Section 2.1.3.4 explains the BERT architecture in details.

2.1.3 Transformers

The transformer is a deep learning architecture [74] which uses stacked self-attention and point-wise. It explores fully connected layers for both the encoder (cf. Subsection 2.1.3.1) and decoder (cf. Subsection 2.1.3.2), as shown in the left and right halves of Figure 2.2, respectively. The Encoder takes an input sequence and maps it into a higher dimensional space (n -dimensional vector). That vector is fed into the Decoder, which turns it into an output sequence. According to Cheng *et al.* [11], self-attention, also called intra-attention, is an attention mechanism relating different positions of a single sequence of tokens to compute a final representation of the sequence. Point-wise consists of two fully-connected layers with an activation function in between. For instance, Rectified linear activation function (Relu).

Currently, literature has presented solutions based on isolated utterances for intent classification tasks. In our view, there is a key lack from previous studies addressing the context relevance in intent classification. In this sense, our research’s originality emphasizes using different approaches to represent the role of context from utterances in dialog flows.

In the following, we explain in further details the elements of encoder, decoder, embedding and softmax from this architecture.

¹An n -gram is a sequence of n words, for instance: a 2-gram (named bigram) is a two-word sequence of words like “please turn”, “turn your”, or “your homework”; and a 3-gram (a trigram) is a three-word sequence of words like “please turn your”, or “turn your homework”. N-gram models are used to estimate the probability of the last word of an n-gram given the previous words, and assign probabilities to entire sequences [38]

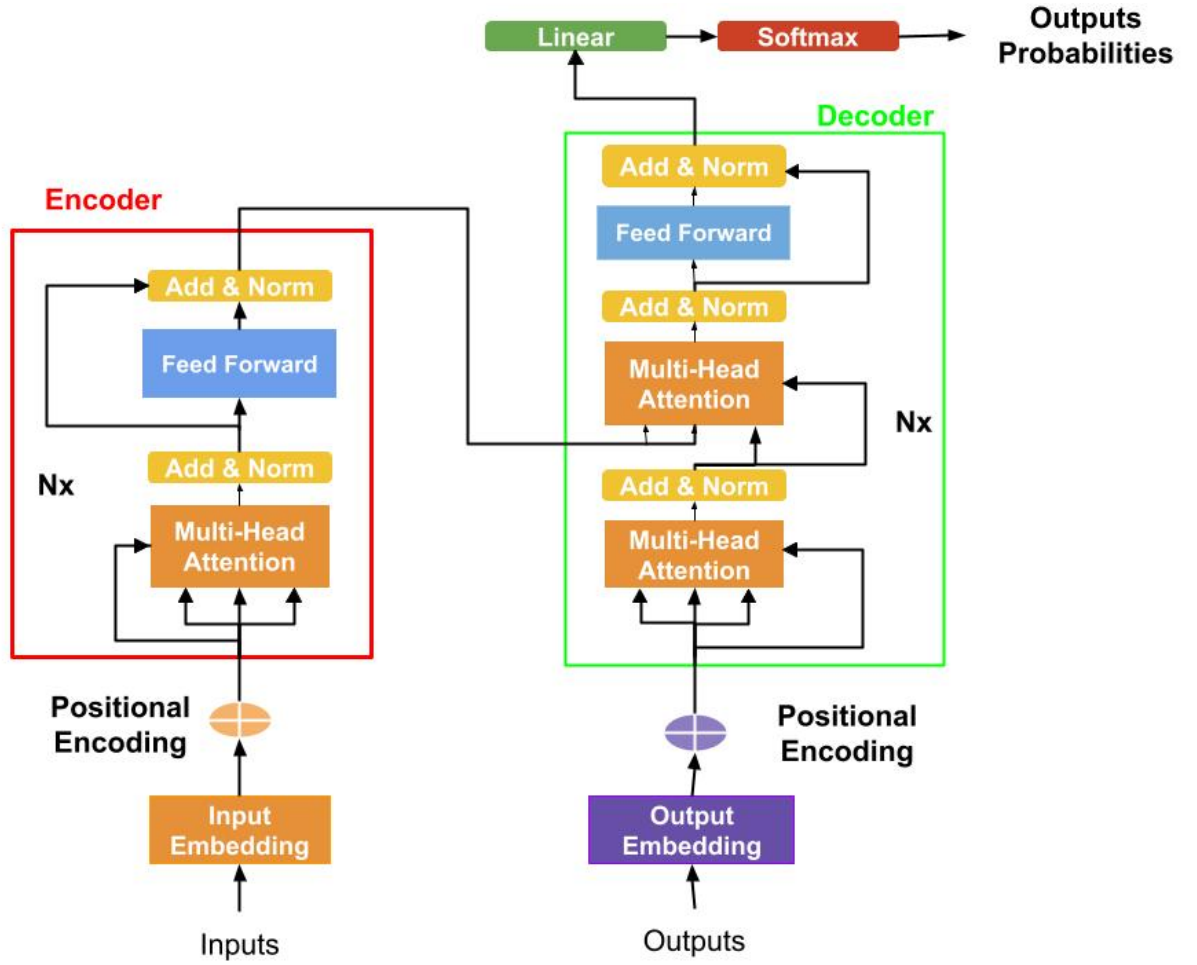


Figure 2.2: Transformer model architecture. Adapted from Vaswani *et al.* [74].

2.1.3.1 Encoder

The Transformer begins by embedding tokens of the input phrase into an embedding matrix. It requires to represent position-based information in the model. Hence, it adds a positional encoding to this embedding matrix. Each encoder layer is composed of two sub-layers, as follows:

1. **Multi-head attention mechanism.** The first sub-layer is a multi-head attention mechanism. Essentially, the Multi-Head Attention refers to several attention layers stacked in parallel, with different linear transformations of the same input.

It consists of four parts: first, linear layers are split into heads. Second, Scaled dot-product attention. Third, Concatenation of heads. Finally, linear layer (cf. Figure 2.3 (Right)). The Linear layer is a simple fully connected neural network that projects the vector produced by the stack of decoders, into a much larger vector.

Each multi-head attention block gets three inputs: Q (query), K (key), and V (value). These are put through linear (Dense) layers and split up into multiple heads. Q is a matrix that contains the query (vector representation of one word

in the sequence); K is all the keys (vector representations of all the words in the sequence); and V is the values, which are the vector representations of all words from the sequence. $\text{Attention}(Q, K, V)$ (cf. Equation 2.1) is called “Scaled Dot-Product Attention” (cf. Figure 2.3 (Left)). The input consists of queries and keys of dimension d_k , and values of dimension d_v . Dot products are computed regarding the query with all keys, divide each by $\sqrt{d_k}$. Then is applied a softmax function to obtain the weights on the values. In practice, attention function is computed on a set of queries simultaneously. The keys and values are packed together into matrices K and V . Equation 2.1 defines the way the matrix of outputs is computed:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2.1)$$

where K^T represents the transposed matrix. The attention output for each head is then concatenated, and finally, put through a final linear layer.

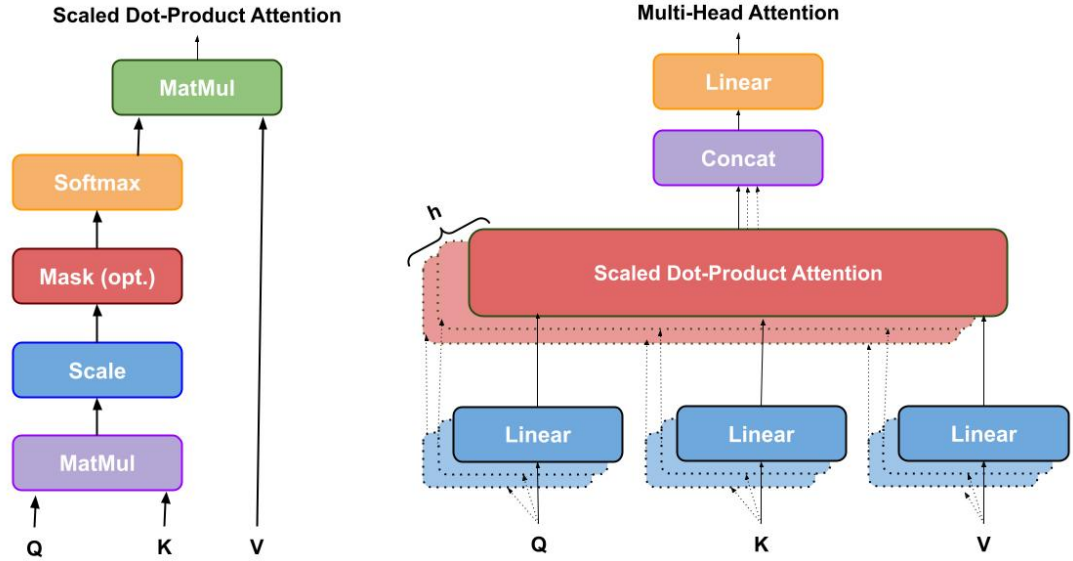


Figure 2.3: Scaled Dot-Product Attention (Left). Multi-Head Attention (Right) which consists of several attention layers running in parallel. Adapted from Vaswani *et al.* [74].

2. **Feed-forward network.** The second sub-layer of the encoder is a position-wise fully connected feed-forward network. Vaswani *et al.* [74] employed a residual connection around each of the two sub-layers, followed by layer normalization (cf. Figure 2.2).

Figure 2.4 presents further details of the encoder. The output of each sub-layer is $\text{LayerNorm}(x + \text{Sublayer}(x))$, where $\text{Sublayer}(x)$ is the function implemented by the sub-layer itself. To facilitate these residual connections, all sub-layers in the model, as well as the embedding layers, produce outputs of dimension $d_{\text{model}} = 512$ (this dimension was defined by Devlin *et al.* [18]). In Figure 2.4, for instance, X_1

and X_2 are any word embeddings of *Thinking* and *Machines*, respectively. Z_1 and Z_2 refer to the outputs of the multi-head attention. The transformer encoder is composed of a stack of $N = 6$. According to Vaswani *et al.* [74], this amount is nothing magical, but it is a standard when modeling transformers identical layers.

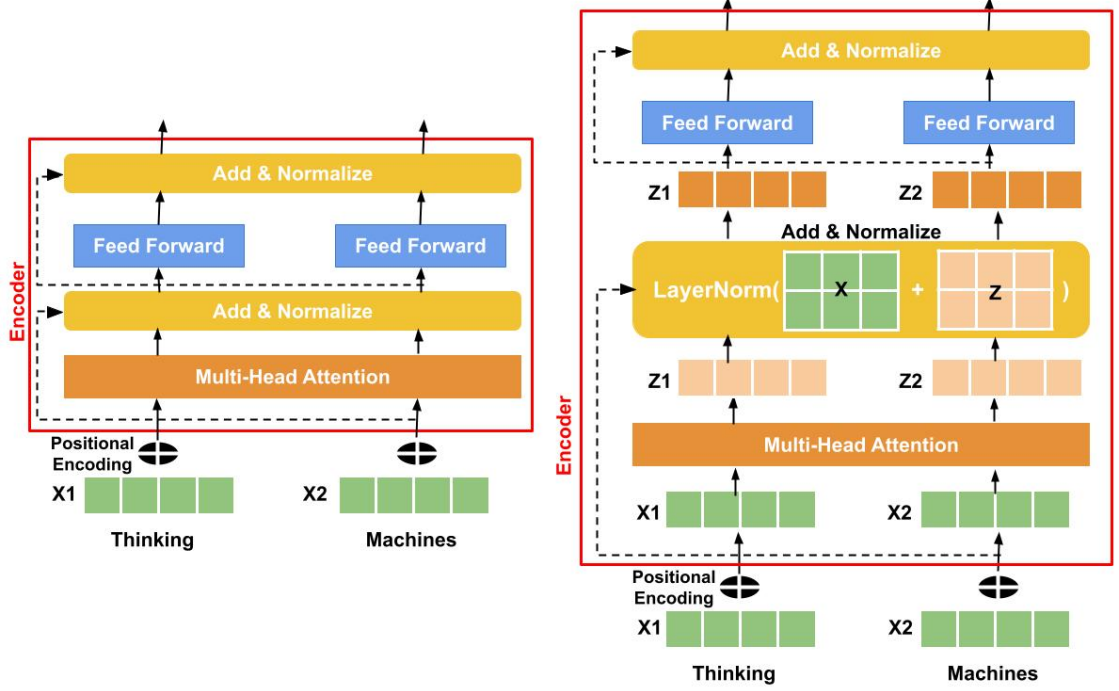


Figure 2.4: Encoder steps are shown in the left of this figure. Particularities of these steps are shown in more details in the right side of the figure. Adopted from Alammari [1].

2.1.3.2 Decoder

The decoder is composed of a stack of $N = 6$ identical layers [74]. In addition to the two sub-layers as in each encoder layer, the decoder inserts a third sub-layer, which performs multi-head attention over the output of the encoder stack. Similar to the encoder, residual connections employ around each of the sub-layers, followed by a normalization layer.

In terms of encoder-decoder, the *query* is usually the hidden state of the decoder. The *key* is the hidden state of the encoder, and the corresponding *value* is the normalized weight, representing how much attention a key gets. The output is calculated as a weighted sum. The dot product of query and key is used to get a value.

2.1.3.3 Embeddings and Softmax

Embeddings are used to convert the input tokens and output tokens to vectors of dimension d_{model} . It uses the usual learned linear transformation and softmax function to convert the decoder output to predicted next-token probabilities [74]. In this model, the

same weight matrix is shared between the two embedding layers and the pre-softmax linear transformation. In the embedding layers, there are multiplied those weights by $\sqrt{d_{model}}$.

There are several types of pre-trained based on transformers, as well as BERT, Roberta, Gpt-3, etc. Some there will show in the next section.

2.1.3.4 BERT: Bidirectional Encoder Representations from Transformer

The BERT (Bidirectional Encoder Representations for Transformers) architecture [18] is a multi-layer bidirectional encoder based on the original Transformer model [74]. BERT is a trained Transformer encoder stack. Basically, BERT received billions of sentences at training time. The model is pre-trained with two strategies on large-scale unlabeled text, for instance, masked language model and next sentence prediction. According to Devlin *et al.* [18], BERT is a deep learning model that has given state-of-the-art results on a wide variety of natural language processing tasks. We selected BERT in this work because it is the most widely used model in NLP research recently.

The input representation for BERT is a summing of three types of embeddings. Figure 2.5 presents a visualization of the input embeddings as the sum of token embeddings, the segmentation embeddings and the position embeddings. The output refers to a vector representation for each token.

- **Token embedding.** This embedding expects to differentiate tokens using context. Thus, it is expected to generate different vectors for equal tokens from context.
- **Segment embedding.** This embedding expects sentence pairs, using 1s and 0s to distinguish between the two sentences.
- **Position embedding.** This embedding needs to learn and use positional embeddings to express the position of words in an utterance.

These embedding are used to feed the BERT model, where given an input token sequence $x = (x_1, \dots, x_N)$, BERT's output is a sequence of hidden states $H = (h_1, \dots, h_N)$. Figure 2.6 illustrates a high-level view of the model. The green blocks correspond to the input token (can have at most 512 tokens); the yellow blocks correspond to the embedding of each token; the purple rectangles correspond to the BERT pre-trained model and a Transformer Encoder stack. The model showed in Figure 2.6 is $BERT_{BASE}$ ($L=12$, $H=768$, $A=12$, total parameters=110M), where the number of layers is denoted as L , the hidden size as H , and the number of self-attention heads as A . The red rectangle corresponds to the hidden state of the first special token ($[CLS]$), and finally, the light blue rectangles are denoted as the hidden state for rest of the tokens.

Input	[CLS]	my	dog	is	cute	[SEP]	he	likes	play	##ing	[SEP]
Token Embedding	$E_{[CLS]}$	E_{my}	E_{dog}	E_{is}	E_{cute}	$E_{[SEP]}$	E_{he}	E_{likes}	E_{play}	$E_{##ing}$	$E_{[SEP]}$
	+	+	+	+	+	+	+	+	+	+	+
Segment Embedding	E_A	E_A	E_A	E_A	E_A	E_A	E_B	E_B	E_B	E_B	E_B
	+	+	+	+	+	+	+	+	+	+	+
Position Embedding	E_0	E_1	E_2	E_3	E_4	E_5	E_6	E_7	E_8	E_9	E_{10}

Figure 2.5: Embedding input representation. The input can represent sentence pairs in one token sequence, for instance, *my dog is cute* (sentence 1) and *he likes playing* (sentence 2). Sentence pairs are packed together into a single sequence. The first token of every sequence is always a special classification token ([CLS]). Then each sentence is separated with a special token ([SEP]) to delimited the sentence pairs. Adopted from Devlin *et al.* [18].

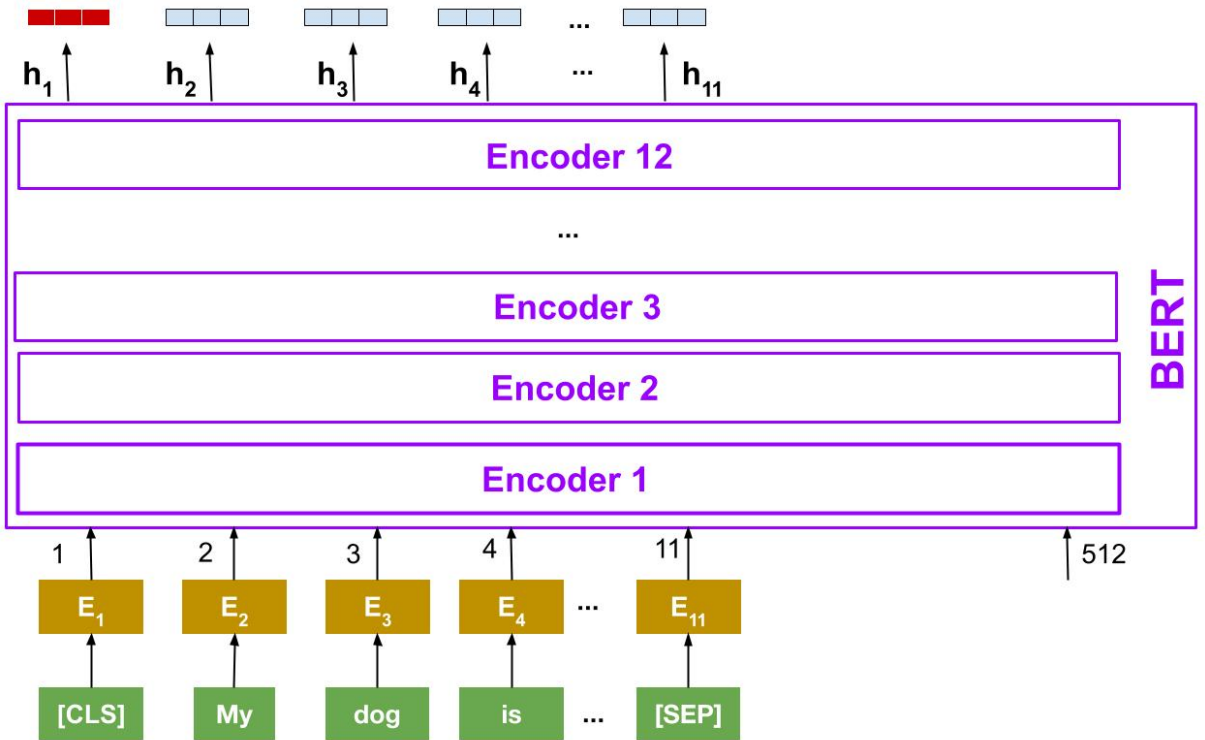


Figure 2.6: A high-level view of BERT model. The input tokens query are “[CLS] my dog is cute [SEP] he likes playing”. Adopted from Chen *et al.* [10].

2.1.4 Transfer Learning

Transfer learning consists in adapting a model trained (*Domain*) on a dataset for a new *task* using the previous knowledge. There are two modalities for transfer learning. The first consists in taking the penultimate output of the pre-trained model to make it the input of a simple classifier. This approach is the feature extraction (FE) method. The

second option consists in fine-tuning (FT) the pre-trained model for the new task [24]. Figure 2.7 presents the learning process of transfer learning.

The transfer learning can be formal defined as follows: given a learned *Task* T_t based on *Source Domain* D_t , it must be able to learn another *task* T_s at the *Target Domain* D_s , where $D_s \neq D_t$ and/or $T_s \neq T_t$. In addition, in most cases, the size of D_t is much larger than the size of D_s [73].

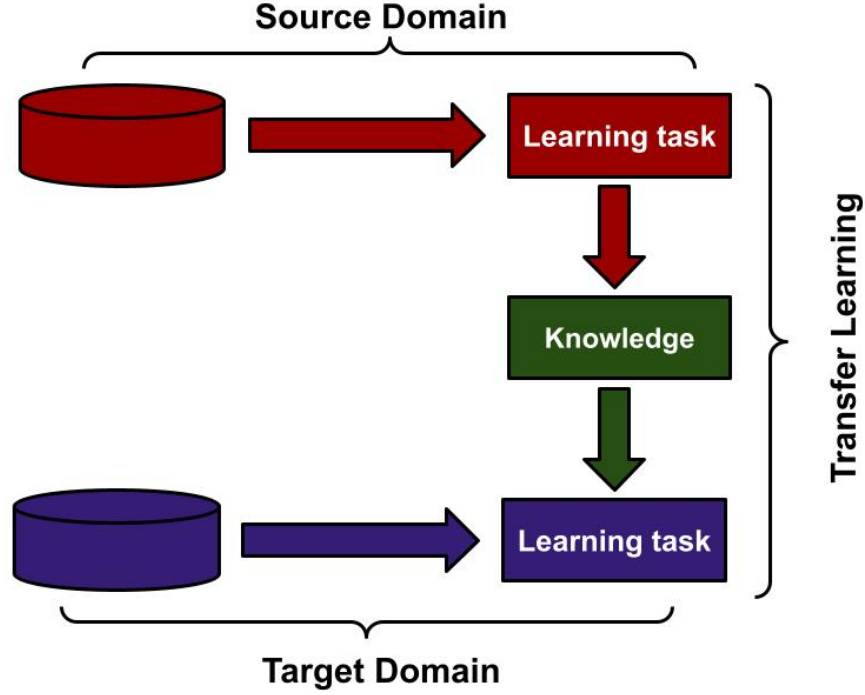


Figure 2.7: Learning process of transfer learning proposed by Tan *et al.* [73]

In this investigation, we use BERT as a source domain and feature extraction approach to get the vector representation of an utterance. Mainly, we used transfer learning because many NLP tasks share common knowledge about language, for instance: linguistic representations and structural similarities. Transfer learning has resulted in SOTA (State of the art) for several supervised NLP tasks [5, 14, 18, 50, 78], as follows: classification, information extraction, Q&A and others.

2.1.5 Active Learning

Active learning is useful when acquiring labels is difficult (expensive, time-consuming, requiring experts, private, etc). This scenario is common for many, if not most, real-world problems [27].

The most popular active learning setting is pool-based sampling [66], where we assume that we have a small labeled dataset L and access to a large unlabeled dataset U from which we choose the samples to be labeled next. We then start an iterative process where in every step the active learning algorithm uses information in L and U to choose the

best $x \in U$ to be labeled. x is then labeled by an oracle and added to L , and the process continues until we have reached the desired amount of samples or test accuracy.

Active Learning aims to select the most useful samples from the unlabeled dataset and hand them over to an oracle (e.g., human annotator) for labeling, so as to reduce the cost of labeling as much as possible while still maintaining performance. This iterative learning approach aims to speed along the learning process, especially if there is not a large labeled dataset to practice traditional supervised learning methods. In Figure 2.8 we described this technique.

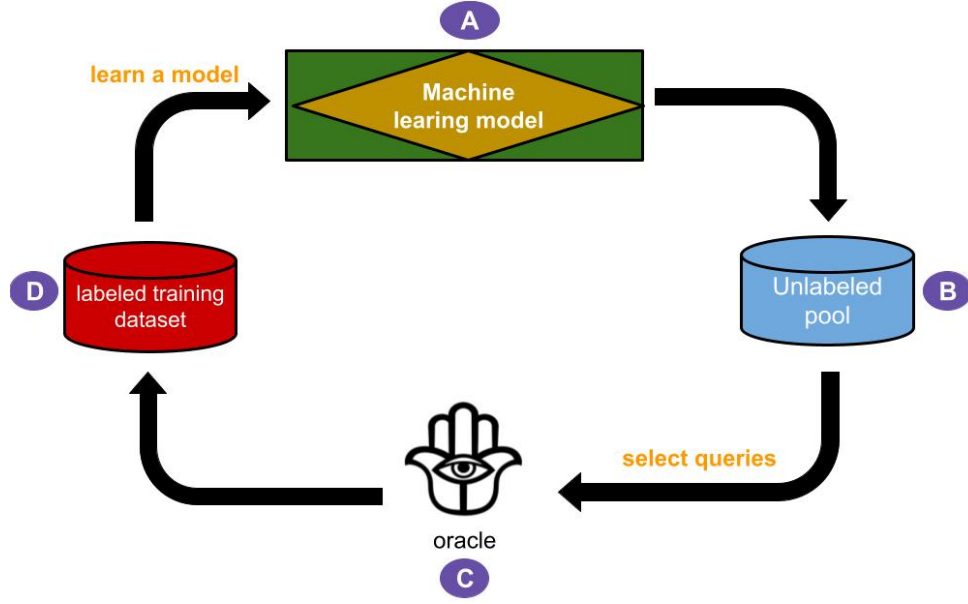


Figure 2.8: Active Learning technique, whose idea is to train and retrain a model in a series of loops. (A) training a classifier with a random selection of samples this has already hand-labeled, (B) predicting labels for the remainder of our data using this model and identify the samples the model was most uncertain about, (C) getting those samples labeled by an oracle, (D) retraining our classifier using the selected random samples, go back to (A) and repeat until there are no more samples without labels.

In our investigation studies how to reduce the number of labeled sample scale and maintain the desired classification effectiveness using Active Learning.

2.1.6 Metrics

In the following, we present the definition of metrics used in this work defined by Gopalakrishna *et al.* [25].

- **Confusion matrix** represents the effectiveness of a predictive model and which classes are being predicted correctly and incorrectly. It presents what type of errors are being made. As an example, Figure 2.9 presents how the four classification metrics are calculated (TP, FP, FN, TN), where:

- **True Positive (TP)**. Observation is positive and is predicted to be positive.

- **False Negative (FN)**. Observation is positive, but is predicted negative.
- **True Negative (TN)**. Observation is negative and is predicted to be negative.
- **False Positive (FP)**. Observation is negative, but is predicted positive.

		Actual Value	
		Positive	Negative
Predicted Value	Positive	TP (True Positive)	FP (False Positive)
	Negative	FN (False Negative)	TN (True Negative)

Figure 2.9: Confusion Matrix for Binary Classification

- **Accuracy** is the proportion of true results among the total number of cases examined.

$$Accuracy = \frac{TP + TN}{TP + FP + FN + TN} \quad (2.2)$$

- **Recall** expresses what proportion of actual Positives is correctly classified;

$$Recall = \frac{TP}{TP + FN} \quad (2.3)$$

- **Precision** expresses what proportion of predicted Positives is truly Positive;

$$Precision = \frac{TP}{TP + FP} \quad (2.4)$$

- **F1 score** refers to a normalized number (between 0 and 1) as the harmonic mean regarding precision and recall.

$$F1 = 2 * \frac{Precision * Recall}{Precision + Recall} \quad (2.5)$$

2.2 Related Work

Intent classification has gradually become a hot research topic in natural language processing. However, to the best of our knowledge, intent classification tasks in Portuguese language have not yet been carefully investigated. According to Zhang [79], methods of

intent classification range from machine learning to knowledge-based models. The state-of-the-art research on intent classification relies on neural network solutions, like Recurrent neural networks (RNN), Convolutional Neural Networks (CNN) and Transformers.

Currently, the main neural networks models available are focused on designing models to fully use the contextual information on an utterance. We reviewed different existing investigations and they were grouped into three different types. First, we reviewed the most used techniques when classifying intent in isolated utterances. Second, we identified which are the most used tools for intent classification. Third, we analyzed proposals that use dialog flow.

Intent classification in isolated utterances. Shridhar *et al.* [67] used Semantic Hashing as embedding for the task of Intent Classification and achieved state-of-the-art effectiveness on three frequently used benchmarks: AskUbuntu, Chatbot and Web Application. Kamphaug *et al.* [39] proposed a deep learning architecture for content recognition that consists of multiple levels of gated recurrent units (GRUs). The architecture was designed to capture a complex sentence structure at multiple levels of abstraction, seeking content recognition for very wide domains, through a distributed scalable representation of content.

Nigam *et al.* [57] developed a methodology to predict intention and slot filling of a query for a chatbot that answers career-related queries. The model used classify intent of a user-query into categories and subcategories using RNNs, and Bi-LSTM with attention approaches for joint slot filling.

Wang *et al.* [75] presented an attention-based encoder-decoder neural network model for joint intent detection and slot filling, which encodes sentence representation with hybrid Convolutional Neural Networks and Bidirectional Long Short-Term Memory Networks (CNN-BLSTM). The proposal decodes it with an attention-based recurrent neural network with aligned inputs. Experiments showed that their model which combines the advantages of BLSTM and CNN can achieve better effectiveness than baseline models.

Howard [35] proposed a fine-tuning study (it groups an extensive set of techniques to find the best possible hyperparameters for the algorithms) of universal language model (ULMFiT), a transfer learning method to be applied to any task in NLP. Finally, the most recent work is Chen *et al.* [10]. Who proposed a joint intent classification and slot filling model based on BERT, aiming at addressing the poor generalization capability of traditional NLU models. Experimental results showed that their proposed joint BERT model outperforms BERT models, modeling intent classification and slot filling separately.

Tools for intent classification. According to Liu *et al.* [49], none of the publicly available Natural Language Understanding (NLU) toolkits, such as Dialogflow², LUIS³ and Rasa⁴, use dialogue context for Intent classification and NER⁵. This is a significant

²<https://dialogflow.com>

³<https://LUIS.com>

⁴<https://rasa.com/>

⁵Named-entity recognition (NER) is a subtask of information extraction that seeks to locate and classify named entities mentioned in unstructured text into pre-defined categories such as person names,

limitation in understanding elliptical or fragment utterances, which depend on the context for their interpretation. Their results showed that on Intent classification, Watson⁶ significantly outperformed other platforms, namely Dialogflow, LUIS and Rasa, although the latter also performed well. This dialogue system used the Dialog State Tracking (DTS) to accurately estimate a compact representation of the current dialog status.

Dialogue flow. Several researches have explored dialog flow to improve the accuracy in different tasks. Khanpour *et al.* [41] applied a deep stacked Long Short-Term Memory (LSTM) with pre-trained word embeddings to classify dialogue acts (DAs) in open-domain conversations. The main reason for stacking LSTM cells is to gain longer dependencies between terms in the input chain. From this proposal, Kumar *et al.* [44] proposed hierarchical Bi-LSTM CRF model also to classify dialogue acts, where the input is a conversation C_i consisting of R_i utterances $u_1, u_2, \dots, u_{R_i}$, with each utterance u_j itself being a sequence of words $w_1, w_2, \dots, w_{S_j}$. This architecture has four main layers, embedding, utterance encoder, conversation encoder, and Conditional Random Fields (CRF) classifier. The output is a dialogue act prediction for each utterance in the conversation.

In the same line, Raheja [60] developed a model with context-aware self-attention, which involves three main components: (i) an utterance-level RNN that encodes the information within the utterances at the word and character-level; (ii) a context-aware self-attention mechanism that aggregates word representations into utterance representations; and (iii) a conversation-level RNN that operates on the utterance encoding output of the attention mechanism, followed by a Conditional Random Fields (CRF) layer to predict utterance labels. In the same research direction, Qin *et al.* [59] proposed a Deep Co-Interactive Relation Network (DCR-Net) to explicitly consider the cross-impact and model the interaction between act recognition and sentiment classification tasks. It takes the dialog act D and sentiment representations S as inputs and then outputs their updated versions which consider cross-impact on two tasks. DCR-Net consists of three components: a shared hierarchical encoder; a stack of co-interactive relation layers that repeatedly fuse dialog act and sentiment representations to explicitly model the relation and interaction between the two tasks; and two separate decoders for dialog act and sentiment prediction.

As far as dialogue state tracking task is concerned, whose primary mission is the identification of all existing slots in an utterance, Gulyaev *et al.* [26] proposed a GOaL-Oriented Multi-task BERT-based dialogue state tracker (GOLOMB) inspired by architectures for reading comprehension question answering systems. The model queries dialogue history with descriptions of slots. This allowed to transfer slot values in multi-domain dialogues and have a capability to scale to unseen slot types. Similarly, Wu *et al.* [77] proposed a contrastive objective function to simulate the response selection task. This pre-trained task-oriented dialogue BERT (TOD-BERT) outperforms strong baselines like BERT on four downstream task-oriented dialogue applications, including intention recognition, dialogue state tracking, dialogue act prediction, and response selection. TOD-BERT con-

organizations, locations, medical, etc.

⁶<https://watson.ibm.com>

catenate all the utterances in the same dialogue into one to capture speaker information and the underlying interaction behavior in dialogue. In the same way, Chao [8] proposed BERT-DST, an end-to-end dialogue state tracker which directly extracts slot values from the dialogue context. BERT-DST used BERT to identify slot values from their semantic context. Also, they used the system utterance from the previous turn and the current turn user utterance as dialogue context input.

Table 2.1 presents a summary of previous studies and our proposal. We reviewed different investigations and grouped them into several tasks. We introduce their respective architecture and explored database. Also, we verified if the approach used the context for its objective task. Finally, we highlight in red color our proposal.

Table 2.1: Comparative analysis regarding existing literature and our proposal.

Task	Architecture/Framework	Dataset	Context
Intent Classification	RNN + Bi-LSTM [57]	ATIS	×
	SemanticHashing [67]	AskUbuntu	×
	GRU [39]	ung.no	×
	CNN-BLSTM [75]	ATIS	×
	BERT [10]	ATIS and Snips	×
Named-entity recognition	Dialogflow [49]	NLU-Evaluation-Data	×
	LUIS [49]	NLU-Evaluation-Data	×
	Rasa [49]	NLU-Evaluation-Data	×
	Watson [49]	NLU-Evaluation-Data	✓
Dialogue acts	Bi-LSTM CRF [44]	SwDA and MRDA	All information
	DCR-Net [59]	Mastodon and Dailymail	All information
Dialogue State Tracking	GOLOMB [26]	Schema-Guided Dialogue	All information
	TOD-BERT [77]	MWOZ	All information
	BERT-DST [8]	MWOZ and DSTC2	last system reply
Intent Classification (Our work)	BERT + Active Learning	ATIS and WvGD	×
	BERT + Dialogue Context	WvGD	several combinations

Currently, literature has presented solutions based on isolated utterances for intent classification tasks, solutions without a standard to select the context for dialogue acts, and dialogue state tracking tasks. In our view, there is a key lack from previous studies addressing the context relevance in intent classification, and the most relevant previous information. In this sense, our research’s originality emphasizes using different approaches to represent the role of context from utterances in dialog flows.

Chapter 3

Investigating Active Learning for Intent Classification on Isolated Utterance

This chapter demonstrates the relevance of getting the best representation for an utterance in the Brazilian Portuguese language. On this basis, we identify the most suitable classifier for intent classification task on isolated utterances. In the same line, we investigate the use of new approaches to address limited labeled data to obtain the training of accurate models.

The remainder of this chapter is organized as follows. Section 3.2 presents our methodology; Section 3.3 presents our conducted experiments including the experimental settings, explored datasets and the experimental results; Section 3.4 discusses our findings; Section 3.5 draws conclusions.

3.1 Introduction

Conversational systems are usually developed for unstructured and open-domain conversations. Various applications are applied, e.g., chatbots, virtual assistants, information retrieval systems, and others. Two factors play a central role in conversational systems: representing an utterance and the amount of labeled data. There is a lack of studies in the Brazilian Portuguese language that investigate classical embeddings and their use with one of the most widely explored model in NLP research nowadays, BERT. Intent classification task has been studied for a long time (cf. Chapter 2). Currently, the primary methods for this task are based on supervised learning [10]. In this context, a significant disadvantage of supervised learning is that it requires many labeled samples for its training process.

In this study, we conduct extensive evaluations to compare several classical embeddings Word2Vec [55], Glove [58], FastText [3] and BERT [18] pre-trained model with the aim of identifying the best representation for an utterance. Similarly, in our study context, we evaluate and compare the effectiveness of different classifiers, namely Support Vector Machines (SVM), Logistic Regression (LR), Neural Network (NN), and a convolutional

neural network (CNN) with a softmax layer for classification.

In addition, our investigation studies how to reduce the number of labeled sample scale and maintain the desired classification effectiveness. To this end, we use active learning (AL) [36] as a semi-supervised machine learning technique that minimizes the amount of labeled data required to build accurate prediction models. Currently, literature has carried out studies addressing classification tasks using active learning. However, we observe a key lack because active learning was not still explored for intent classification tasks on a Brazilian Portuguese corpus.

There are publicly available datasets for intent classification tasks as, for instance, ATIS [31]. Messages in these datasets contain low variance and a limited number of classes compared to the number of classes presented in real-world applications. In our study, we use ATIS to compare our model with previous results in literature. Furthermore, we carry out a validation based on complex data from real-world applications based on a new dataset containing conversations in Portuguese language between humans and chatbots. The chatbots were built on the Dialogflow platform and the messages were in the Brazilian Portuguese language. This dataset contains message outliers, language slangs, several grammatical errors, and several intent classes.

According to Lu *et al.* [51], one crucial component of supervised learning and active learning for text classification tasks is the mechanism used for text representation. Several methods in literature use the bag-of-words technique (BOW) to obtain a vectorial representation of the text. Afterwards, they explore some supervised learning and active learning technique, *e.g.*, [36, 68].

Another way to capture semantics in a fixed vector length is using word embedding. Over the past few years, studies applying word embedding to the task of intent classification have achieved noteworthy success [63, 70]. Some standard approaches to learn word embeddings are Word2Vec [55], Glove [58], FastText [3]. Current state-of-the-art word embedding uses neural network architectures based on Transformer [74] that have been pre-trained, like BERT [18], Xlnet [78], Roberta [50], Electra [14] and GPT-3 [5]. In this work, we selected BERT.

In this chapter, experimental results show that our method based on pre-trained models and convolutional neural networks using an active learning approach achieves success for intent classification on utterances in Brazilian Portuguese than classical embeddings. Our results achieve equal or superior accuracy than traditional supervised learning approaches with more than half of training data. Also, we obtain accurate results with less than half of data in small datasets such as ATIS.

3.2 Methodology

We propose an active learning approach for intent classification using different message representation mechanisms. The method consists of four stages: pre-processing, compute

vectorial representations, classification, and an active learning module. Figure 3.1 presents our methodology using active learning and a convolutional neural network with a softmax classifier (after FC4).

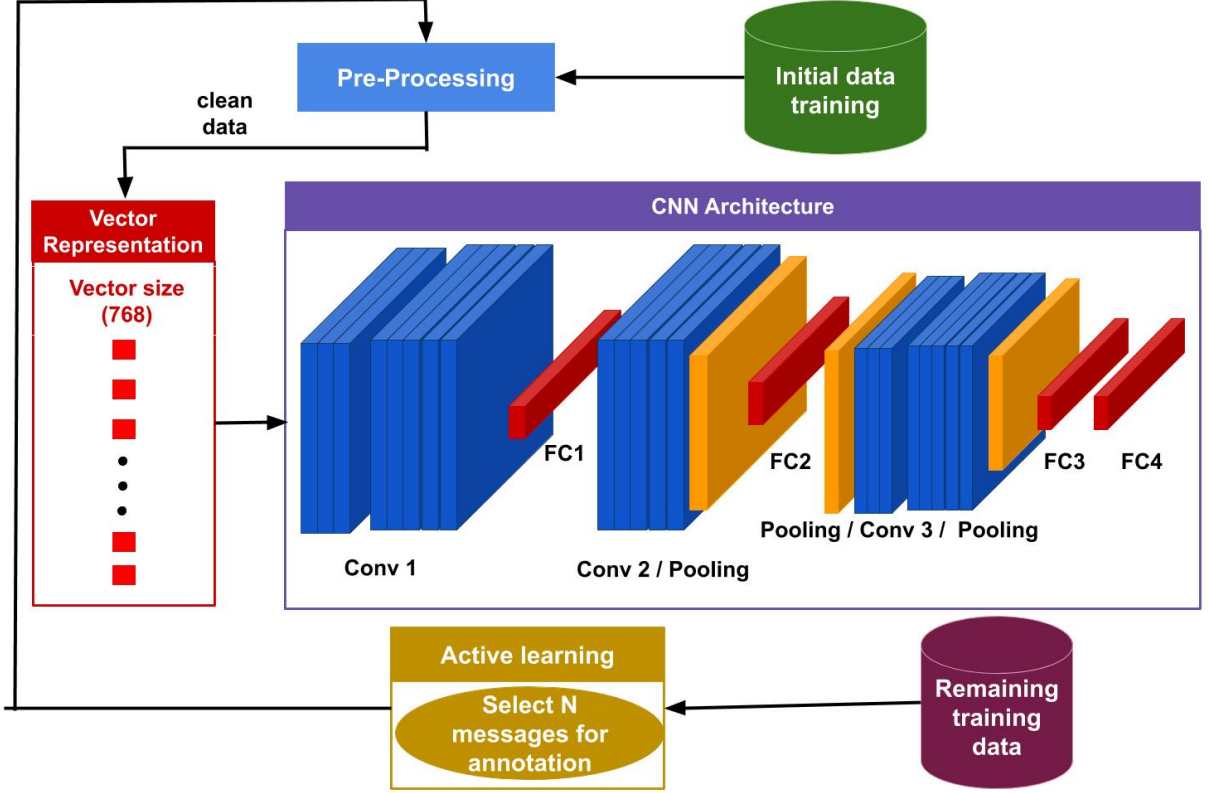


Figure 3.1: Active learning approach with a convolutional neural network architecture with softmax for classification. The preprocessed message feeds the vector representation module that outputs 768-dimension feature vectors (BERT). One-dimension convolution and pooling are applied (Conv1, Conv2/Pooling, and Pooling/Conv3/Pooling). The result is flattened and concatenated in a feature vector (FC4) until the final decision-making stage. Then, the active learner module selects a sample from the remaining training data and carries out active learning algorithms such as uncertainty [6] or ranked batch [51]. Inspired from Zhang [79].

In our methodology, we used a traditional supervised learning approach as a baseline method. This process consists of three stages (cf. Figure 3.2 shows the overall supervised learning approach). First, the system takes all input messages and cleans each utterance in the pre-processing stage. Second, each message is transformed to its vectorial representation using word embedding. Finally, the vector representation obtained is used to train a classifier.

3.2.1 Pre-processing

A message contains unnecessary information which can be removed. We apply preprocessing steps to clean input queries. The pre-processing step includes lower-casing, removing

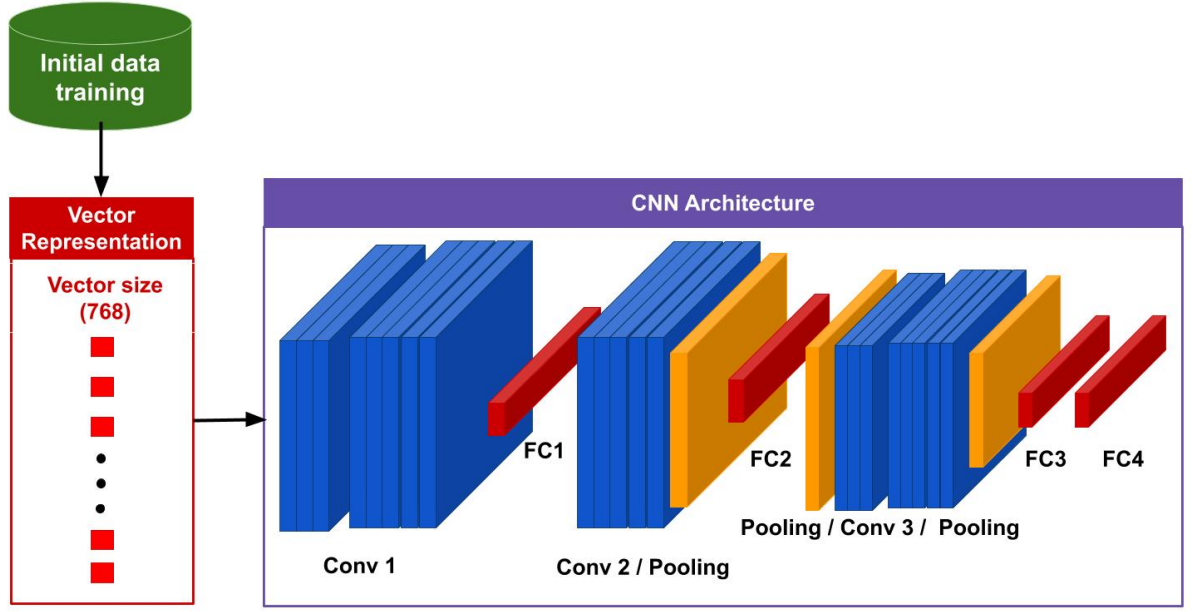


Figure 3.2: Supervised learning approach using a convolutional neural network architecture with softmax for classification. The preprocessed message feeds the vector representation module that outputs 768-dimension feature vectors (BERT). For this vector, one-dimension convolution and pooling are applied (Conv1, Conv2/Pooling, and Pooling/Conv3/Pooling), and the result is flattened and concatenated in a feature vector (FC4) until the final decision-making stage.

punctuation on the corpus, removing URLs, removing stop words and the message is finally a vector of tokens so that it can be fed to the vectorial representation-model, for instance: For the utterance *'Já paguei o boleto da campanha ## porque ele ainda está pendente no site <https://campanha.com/5cb4abba34070929d959d32d?>'* its correspondent output is *'paguei boleto campanha porque ainda pendente'*.

3.2.2 Compute Vectorial Representation

Vectors of tokens obtained by using pre-processing are used to compute vectorial representations. The embedding of a token is a vector, and each element in the vector is a real number to represent one dimension feature of the word. The embedding vector refers to a feature set of the word in a low-dimension space, and indicates latent information of the word.

Word embeddings aim to get the distributed representation of each word. For a message tokenized, for each token t_i , we first represent it as an embedding matrix $E_i \in \mathbb{R}^{D \times V}$, where D is the dimension of word embeddings and V is the dimension of the word dictionary than we obtain the embedding of the token $e_i t_i$. In this context, e_i is the embedding of each token t_i . In this sense, we get each word embedding, and the matrix E_i consists of all the embeddings $e_i t_i$ for words in the dictionary, which is randomly initialized and trained during the training process.

In this work we used pre-trained embedding models provided by Hartmann *et al.* [28] such as Word2vec, FastText and Glove because these embeddings were built using a large database in Brazilian Portuguese. Each model has CBOW and Skip-Gram variations; except Glove, these variations generated different token predictions. We use CBOW variation because this variation got the best results during experiments carried out in [28].

We carry out two different vectorial representations. In addition to classical embeddings, our work explored the BERT as embedding in our study context. In an experimental perspective, we aim to understand to which extent the use of BERT for representing utterances improve the classification results compared to the classical embeddings. In BERT, results of an Encoder are vector representations with 768 for each token. These operations are performed for each Encoder layer (cf. Section 2.1.3). Finally, the representation vector of the sequence is obtained at different hierarchical levels using the different Encoders of BERT. Specifically, we use the output generated by the [CLS] token and the dimensional vector representation obtained at the last layer of BERT model as a vector representation for the sequence.

3.2.3 Classification

Vector representations obtained by using classical embeddings or BERT can be used to train a classifier, like for example, support vector machine, multilayer perceptron, or a convolutional neural network with softmax function (cf. Figure 3.1). In this context, vector representations are the input for a convolutional neural network. In particular, our investigation uses a convolutional neural network because this model can perform low and fast computational computations compared to RNN and Transformers.

We carry out eight one-dimension (1D) convolutional filters. The results of all these convolutions are flattened. After there is a sequence of 1D convolution, average-pooling, and fully-connected layers. According to Schwarz *et al.* [64], the rationale behind this structure is to capture intrinsic syntactic and semantic patterns. Finally, *FC3* contains 120 units in the hidden layer and *FC4* presents 66 units (number of classes in the Brazilian Portuguese dataset) in the last layer. Then, a softmax layer was used to output the probabilities of the sequence belonging to each of the interest classes. We used the Adadelta optimizer and mean squared error as loss for updating the weights to evaluate the effectiveness of this classifier. The system was trained for 50 epochs to obtain weights for the classification model. After carrying out several preliminary experiments, we identified the best hyper-parameters settings to CNN architecture (cf. Table 3.1).

3.2.4 Active Learning Module

The active learner module in our proposal selects a sample from the remaining training data and carries out active learning algorithms in the training process. Most used algorithms in literature which are used in our study included: 1) uncertainty [6]; and 2)

Table 3.1: Convolutional Neural Network hyper-parameters.

Layer Name	Description	#filters	filter size	stride	#units
Conv1	1D Convolution	8	(1×3)	2	-
FC1	Fully-connected	-	-	-	384
Conv2	1D Convolution	12	(1×5)	2	-
Pooling	Average-pooling	-	(1×5)	2	-
FC2	Fully-connected	-	-	-	192
Pooling	Average-pooling	-	(1×5)	1	-
Conv3	1D Convolution	15	(1×3)	1	-
Conv3	1D Convolution	15	(1×5)	3	-
Conv3	1D Convolution	15	(1×7)	5	-
Pooling	Average-pooling	-	(1×3)	2	-
FC3	Fully-connected	-	-	-	100
FC4	Fully-connected	-	-	-	66

ranked batch [51].

This technique requests their labels from the dataset. We repeat this process until no more samples are available in our training set. The module in each iteration takes the sample with the lowest confidence (we compute this confidence using an uncertainty technique) as the most informative one; it selects such a sample and retrieves its label from the dataset.

The uncertainty approach assigns an uncertainty value to each sample [6], which defines how uncertain the classifier is in finding the correct label for the sample. The most uncertain samples are then chosen to be labeled using heuristics. The simplest measure is the uncertainty of classification defined by the equation $U(x) = 1 - P(\hat{x}|x)$, where x is the instance to be predicted and $\hat{x}$ is the most likely prediction. Then, the algorithm computes the uncertainty and we compare this value with a threshold (cf. Figure 3.3 (a)). If this value is lower than the threshold, we used the sample to train a new model, otherwise, the sample was give back for remaining data (cf. Figure 3.3 (b)).

Ranked batch approach [7] to active learning is described as follows: let D be a training set containing instances in the equation $d_i = x_i, c_i$, where x_i is a vector containing values for each attribute and $c_i \in C$ a categorical feature indicating its class. Let U be an unlabeled set consisting of instances in the equation $u_i = (x_i)$, *i.e.*, only the attribute values are known. The ranked batch approach generates a ranking Q , using information of D , containing instances of U so that they should be labeled aiming to maximize the effectiveness of a given classifier. For instance, Figure 3.4(a) presents that the new input to train the new model corresponds to a list of Q lower values. In the same way, Figure 3.4(b) shows the iterative application of ranked batch approach.

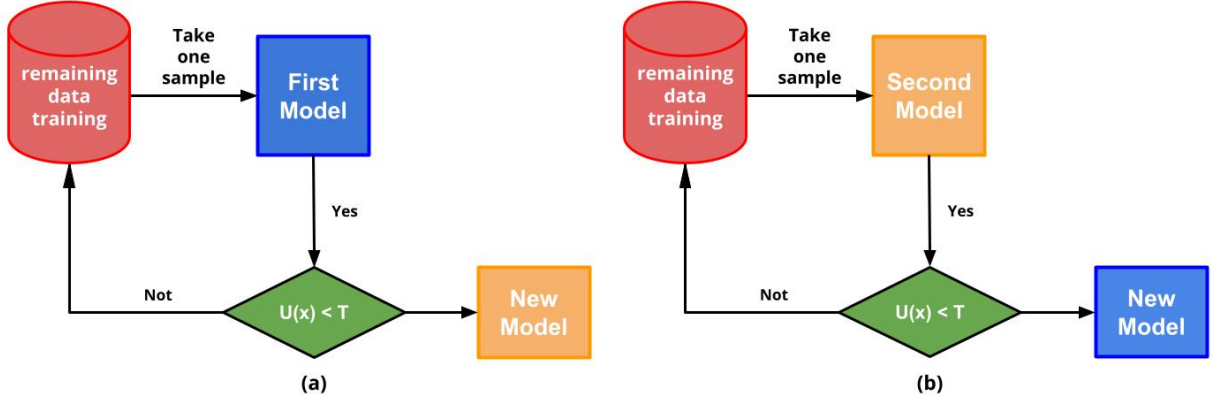


Figure 3.3: Uncertainty strategy employed with two different stages (from left to right): (a) we used an initial data to train a first model, then we pick up a sample from the remaining data after we compute the uncertainty; (b) if the uncertainty is better than a threshold, we create a new model (second model). We repeat this process until no more samples are available in our training set (remaining data).

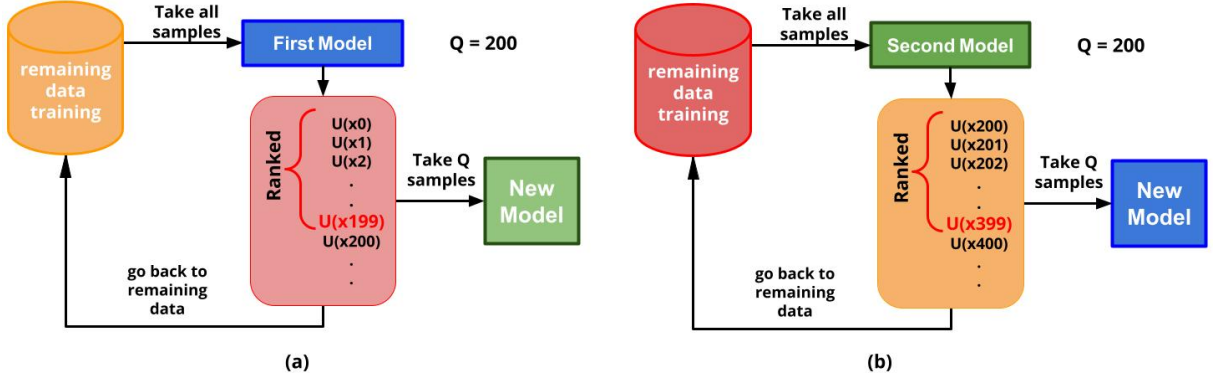


Figure 3.4: Ranked batch strategy employed with two different stages (from left to right): (a) we used a portion initial of data to train a first model. We then compute the uncertainty for the whole remaining data, and pick up the Q lower values to train a new model. The remaining of data is given up to the data training. (b) we update the model and the previous process is repeated until does not have more data.

3.3 Experiments

This section presents extensive experiments to evaluate the supervised learning and active learning approaches addressed in our work on two datasets.

3.3.1 Datasets

For the evaluation of the proposed method, we used two datasets, Airline Travel Information Systems (ATIS) dataset [31] and WaVy Global Dataset (WvGD)¹.

ATIS is a widely used English dataset in spoken language understanding, where audio recordings of people making flight reservations are collected and manually transcribed.

¹<https://wavy.global/es/>

The ATIS (Airline Travel Information System) dataset was collected by DARPA in the early 90’s [31]. It is one of the most frequently used dataset for SLU (spoken language understanding) research. It was based on spoken queries regarding flight-related information. An example utterance is: “I’d like to fly from Boston to San Francisco.” The dataset exists in both spoken and text form; we use the text form here. Table 3.2 presents the statistical information on ATIS dataset. The three first columns in Table 3.3 count the occurrences of train, test, validation set. The fourth column indicates the number of categories (classes).

Table 3.2: Statistical information on ATIS dataset [31].

Dataset name	Train	Valid	Test	#classes
ATIS	4478	500	893	18

WvGD is a new dataset of conversations between humans and chatbots in Brazilian Portuguese. The messages were collected from virtual assistants. The whole dataset has 357.182 utterances. The Wavy enterprise (in Brazil) has created its first cleaning version, which contained 44.520 utterances collected from platforms such as WhatsApp and Facebook. This cleaned version has conversations between people and a chatbot implemented in Dialogflow. This WvGD version was limited in using isolated messages with a confidence greater than 70%, and Dialogflow provided this percentage for each utterance. Table 3.3 shows the number of categories and frequencies of messages in WvGD. The three first columns in Table 3.3 count the occurrences of train, test, and validation set. The fourth column indicates the number of categories (classes).

Table 3.3: Occurrences of messages and number of classes in WvGD dataset.

Dataset name	max	min	mean	stdev	#class
WvGD	1700	111	674	82%	66

Personal information in the WvGD dataset was masked and anonymized. For example, Digits were replaced by the symbol ‘#’, E-mails addresses were replaced by ‘[[E-MAIL]]’, some companies’ names were replaced by ‘[[COMPANY_NAME]]’ and some particular words were replaced by ‘[[CENSORED_WORD]]’.

3.3.2 Experimental Settings

In order to evaluate the effectiveness of our proposal, we randomly divided the WvGD into 40% of samples to train the classifier and 60% for testing it. This distribution is because we aimed to simulate the distribution of data in real-world applications. From the training set, we selected 2000 messages in each iteration according to the active learning techniques.

On the ATIS dataset, 500 sentences were selected in each iteration. We considered this value because we compared our results with a recent study [79] that uses active learning with the same distribution. Our experiments applied several vector representations including configurations with classical embeddings (FastText (FT), Glove (GV), and Word2vec (WO)) and explored the BERT approach. We used different models for the classification process, namely Support Vector Machines (SVM) [30], Logistic Regression (LR) [4], Neural Network (NN) [34], and a convolutional neural network (CNN) [47] with softmax to a classifier. Experiments used two active learning techniques considering Uncertainty and Ranked batch. For the Brazilian Portuguese language, we utilized the multilingual model named multilingual_L-12_H-768_A-12, and for the English, we employed the pre-trained BERT [18] model (bert-base-uncased)².

Table 3.4 shows a list of parameters for each classifier.

Table 3.4: Parameters for each classifier. We highlight in red color the best parameter after a grid search process.

Classifier	Parameters
SVM	'C': [0.01, 0.05, 0.1, 0.5, 1, 5, 10]
	'loss': ['hinge', ' squared_hinge ']
	'max_iter': [500, 1000 , 5000]
	'tol': [0.00001, 0.0001, 0.001 , 0.01, 0.1]
RF	'C': [1, 1.5 , 2]
NN	'hidden_layer_sizes': [100 , 50, 10]
	'activation': [relu , 'tanh']
	'solver': ['sgd', ' adam ']
	'learning_rate_init': [0.01 , 0.02]
	'max_iter': [100, 500, 1000]

3.3.3 Computational Resources

Our method proposed was implemented in Python programming language due to the large availability of libraries for data manipulation, numerical computation, text processing, and deep learning, such as NumPy, NLTK, Transformers, TensorFlow, Keras, and Pytorch.

The experiments were performed on Kahuna cluster available at State University of Campinas. The cluster is equipped with 28 Graphic nodes with 48 cores HT Intel Xeon E5-2670 v3 of 2.30GHz, 64G of memory, and 2 video cards, Nvidia Tesla K40M.

3.3.4 Results

We compared results obtained with the traditional supervised learning approach and our implemented active learning approach. We carried out combinations between embeddings, classifiers, and active learning techniques. Tables 3.5 and 3.6 present the results using classical supervised learning.

²<https://github.com/google-research/bert>

Table 3.5: Results with supervised learning approach: classification results obtained by using different vector representations and classifiers.

Classifier	WvGD				ATIS			
	Accuracy	Recall	Precision	F1-Score	Accuracy	Recall	Precision	F1-Score
FT + SVM	60.09	53.38	61.30	57.06	87.54	40.37	89.34	55.61
FT + LR	57.06	45.18	58.56	51.01	84.64	23.56	86.82	37.06
FT + NN	62.33	56.69	64.03	60.13	88.39	39.75	90.04	55.15
GV + SVM	60.99	54.99	62.25	58.40	81.40	42.59	82.83	55.25
GV + LR	58.47	47.18	59.56	52.65	79.35	23.55	82.26	36.62
GV + NN	62.51	57.98	63.34	60.54	81.74	45.89	83.95	59.34
WO + SVM	62.23	57.48	63.54	60.36	72.31	55.28	74.39	63.43
WO + LR	61.00	51.75	62.87	56.77	70.27	50.12	72.49	59.26
WO + NN	63.24	59.32	65.34	62.18	74.35	58.34	76.49	66.19

Table 3.6 shows that the pretrained BERT embedding attains the best results compared with other embeddings. It shows that the best results correspond to BERT + CNN. We carried out experiments with the CNN architecture only on BERT because it presented the best results compared to classic embeddings. Our best result is 78.39% for WvGD dataset (considered our baseline) (cf Table 3.6).

Table 3.6: Supervised learning approach: classification results obtained by using BERT representations and classifiers.

Classifier	WvGD				ATIS			
	Accuracy	Recall	Precision	F1	Accuracy	Recall	Precision	F1
BERT + SVM	69.97	67.82	73.23	70.42	95.90	67.00	97.72	79.49
BERT + LR	69.87	59.95	73.54	66.05	95.22	67.11	97.62	79.54
BERT + NN	71.79	68.50	75.92	72.02	96.24	78.83	98.04	87.39
BERT + CNN	78.39	72.52	82.01	76.97	98.20	84.23	98.69	90.89

Figures 3.5 and 3.6 show the results of the uncertainty and ranked batch active learning methods on the WvGD dataset.

Figure 3.5 illustrates the results of the accuracy metric with our method based on BERT + CNN (dotted green lines) and Uncertainty as active learning. It achieved equal or better accuracy than the supervised learning approach (dotted red lines) with more than half of the training data. An increase of accuracy reached 0.68% during iteration 9. The overall accuracy reached 4.19% during iteration 13.

Similarly, Figure 3.6 provides results about our method based on BERT + CNN (dotted green lines) and Ranked batch (active learning). It achieved better accuracy than Uncertainty (Figure 3.5). For instance, during iteration 9 the accuracy reached 0.70%, and during iteration 13, a rise of accuracy reached 4.65% in comparison to the baseline (dotted red lines).

The best result from Table 3.6 is 98.20% for ATIS dataset (considered our baseline). Figures 3.7 and 3.8 provide the results in applying the uncertainty and ranked batch active learning methods on the ATIS dataset.

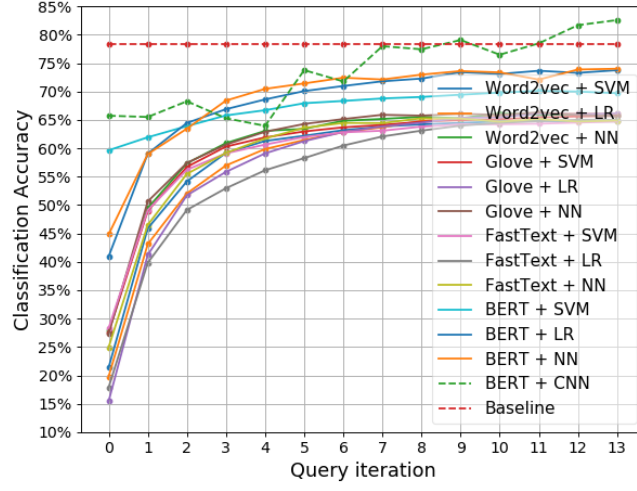


Figure 3.5: Accuracy results using uncertainty method on WvGD dataset.

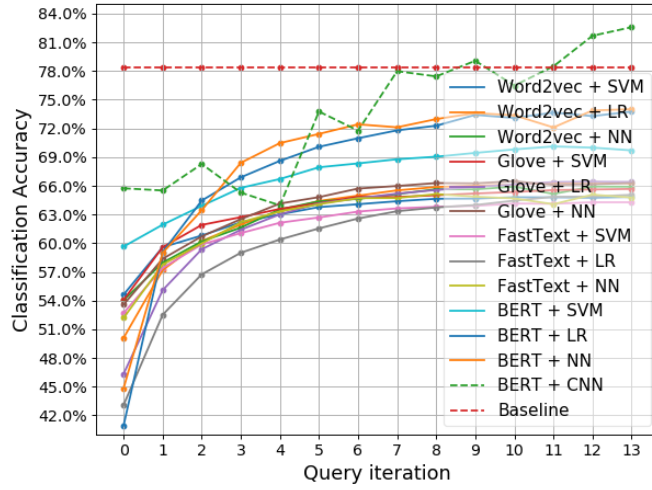


Figure 3.6: Accuracy results using ranked batch method on WvGD dataset.

Figure 3.7 describes the results of the accuracy metric concerning our method based on BERT + CNN (dotted green lines) and Uncertainty (active learning). It also achieved better accuracy than the supervised learning approach (dotted red lines) with less than half of the training data. A growth of accuracy reached 0.25% during iteration 3, and a peak during iteration 5 equals 0.41%.

Likewise, Figure 3.8 reveal the results on our method based on BERT + CNN (dotted green lines) and Ranked batch. It achieved better accuracy than Uncertainty (Figure 3.7). In this case, during iteration 3 the accuracy reached 0.28%, and during iteration 5, an increase of accuracy reached 0.55% in comparison to baseline (dotted red lines).

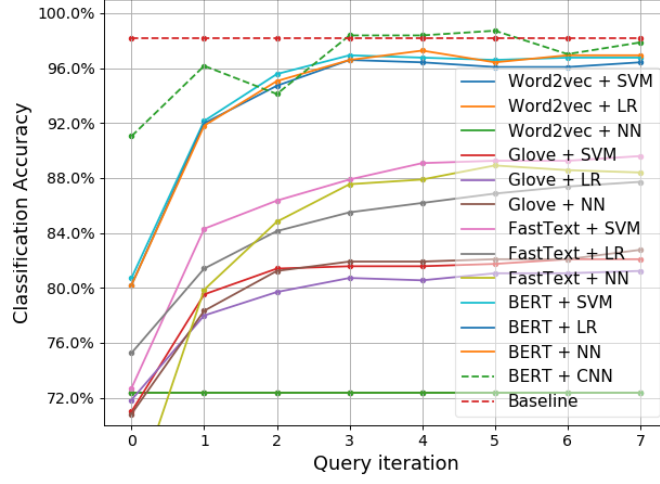


Figure 3.7: Accuracy results using uncertainty method on ATIS dataset.

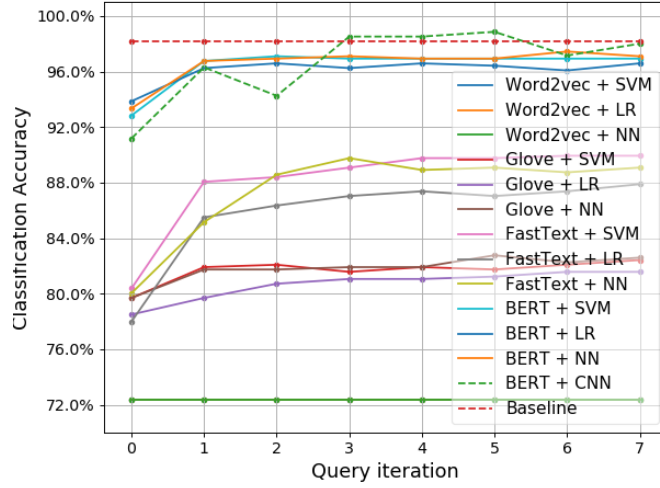


Figure 3.8: Accuracy results using ranked batch method on ATIS dataset.

3.4 Discussion

Our obtained results suggest that the use of classical word embedding (cf. Table 3.5) with active learning techniques is less accurate than the supervised learning approach (baseline method) to carry out message representations. On the other hand, using pre-trained models based on BERT (cf. Table 3.6), we achieved better results than classical word embedding. Nevertheless, according to results from Figures 3.5, 3.6, 3.7 and 3.8, only using a CNN architecture was possible to obtain a higher accuracy than the baseline. We observed that adding layers to a network (convolution and pooling layers) improved the results from these experiments. More specifically, this improvement was because BERT learns noise, and we eliminated this noise by training a convolutional neural network architecture. Consequently, this architecture captures the relevance of the syntactic and

semantic patterns.

Our methodology using active learning is better in contrast to other current results. For instance, [79] created an accurate model carrying out a sophisticated, active learning method (baseline in active learning). However, the improvement of accuracy of our model reaches 0.30% above baseline in active learning. We only used less than half of the training data compared to the work in [79], even though there was a fall in the accuracy. Nevertheless, we used basic models for the active learning approach, and we got an improvement by including a CNN architecture. In this sense, we improved current results by refining the active learning method combined it with the CNN architecture. A limitation of our results corresponds to the need to address the balancing of the data. During the next Chapter 4, we address that problem.

Although the improvement is trivial on ATIS, the advantages of the proposed active learning methods can be substantial because we used only less than half of the training data. In the WvGD dataset, we observed that the accuracy is more significant as the amount of data is more significant. We perceived similar results for the recall metric.

In summary, compared with the traditional supervised learning approach, our proposed method can build equal or better intent classification models with much less training data and reaches higher accuracy as more as our architecture adds more data.

3.5 Conclusion

Intent classification in natural language messages remains a challenging task for computer-supported communication. In this chapter, we presented a method for automatic intent classification in utterances appearing from a chatbot in Brazilian Portuguese language. It is a challenging dataset containing message outliers, language slangs, grammatical errors, and many classes (real-world data set). Achieved results reached equal or better accuracy than traditional supervised learning approaches, needing more than half of training data. We used less than half of the training data to get higher accurate models in small datasets like ATIS. We showed that pre-trained models like BERT are more adequate than classical word embeddings for message representation in the investigated task. We improved our results by adding a CNN architecture. We found that the analyzed active learning techniques minimized the amount of labeled data required to build accurate prediction models and improved accuracy compared to the supervised learning approaches.

Chapter 4

Intent Classification Relying on Dialog Flow Contextual Information

In the previous chapter, we identified the best way to represent an isolated utterance in Brazilian Portuguese [21]. We also found the most accurate classifiers in this context. This chapter investigates the relevance of the context (one or few previous utterances) in the classification of current utterance. We carried out several combinations concerning the application of context. In our experiments conducted, we contrasted the use of context and no-context.

The remaining of this chapter is organized as follows. Section 4.2 presents our methodology to address the problem. Section 4.3 presents how the experiments were conducted. Section 4.4 presents the obtained results and discusses the findings. Section 4.5 draws conclusions.

4.1 Introduction

Within a communication process, people use prior information flow to obtain a conversation context. In the same line, the analysis of an utterance to identify user's intention in a dialogue system can benefit from the conversation context. However, literature on intent classification does not address this significant challenge.

The Dialog State Tracking task [8, 26] uses previous information and gets accurate results tracking the dialogue's current state. On this basis, we consider relevant to study the information context for intent classification in our task.

In this chapter, we propose a novel approach to managing context, a windows-based procedure. Our window-based strategy aims to make use of windows on the previous information of the dialog flow.

We created a new version from WVDS's full version (mentioned in Section 3.3.1). In our previous version, we focused on isolated sentences. However, for our current experiments, this version is not adequate because we need dialog flows. In this study, we carry out experiments to compare intent classification based on isolated utterances and queries

with context. Like the previous version, the current WVDS dataset contains message outliers, and numerous grammatical errors.

Literature has not carried out comprehensive studies using the context in the intent classification task. To the best of our knowledge, there is a lack in how solving this problem. Previously studies in literature did not study the relevance of context during intent classification tasks [10, 63, 79]. Existing studies used only the last utterance [45, 67, 70] to carry out intent prediction. As for other tasks such as Dialogue State Tracking and Dialogue Acts do not standardize the context used. Some works used all previous information, and others used the last system reply.

In our work, we proposed diversified approaches and combinations to use a dialog flow’s earlier conversations for intent classification. These approaches use previous information of a dialog flow to increase the contextual knowledge to improve our model. This architecture makes it possible to train the dialogue history and the current utterance jointly. Our approach is robust to imbalanced datasets due to we modify our loss function to penalize misclassification of the underrepresented classes more. Our results achieved substantial improvements over the baseline in three approaches using the user utterance and system response from previous messages as dialogue context.

4.2 Methodology

Figure 4.1 presents the proposed method. It is mainly composed of four components (dotted lines of different colors delimited each module): (dotted red lines) pre-processing (we use the same procedure presented in Section 3.2.1); (dotted blue lines) dialogue context module; (dotted green lines) feature extraction; and (dotted orange lines) classifier. Figure 4.1 shows a dialog flow pre-processed (red circle, A), then presents several types of trajectories encoded in our solution (blue circle, B). Our model takes a dialogue context and the current utterance as input for each user turn (purple circle, C). The BERT-based encoding module encodes the dialogue context input to produce contextualized sentence-level and token-level representations (green circle, D). The classification module then uses the sentence-level representation (b_0) to generate a categorical distribution over twenty-two types of categories (orange circle, E).

4.2.1 Dialogue Context Module

To capture user information and the system’s response in a dialogue flow, we used two special tokens, $[USER]$ and $[SYSTEM]$. We prefix the special tokens to each user utterance and system response, then concatenated all utterances in the same dialogue into one flat sequence (cf. Figure 4.1). For example, for a dialogue $D = \{U_1, S_1, \dots, U_n, S_n\}$, where n is the number of conversation turns (messages send by the user), and each U_i or S_i contains a sequence of words, the input of the pre-training model is processed as



Figure 4.1: Several types of trajectories (from left to right) in our window-based approach: (a) shows an initial dialog flow; (b) exhibits an approach called *'all context'* that uses the complete previous information (user and bot) as context; Similarly, (c) presents a window-based approach called *'user context'* that employs the user's previous messages to get the context; (d) reveals a window-based approach denominated *'user-bot context'* that employs the last utterance from user and from the bot to get the context; (e) reveals a window-based approach denominated *'last-user context'* that employs the user's last utterance to get the context. Finally, (f) exposes a window-based approach denominated *'last-bot context'* that employs the bot's last utterance to get the context.

“[*USER*] U_1 [*SYSTEM*] S_1 ...” with standard positional embeddings and segmentation embeddings (input defined in Section 2.1.3.4).

We defined five dialogue representation types: *all context*, *user context*, *user-bot context*, *last-user context*, and *last-bot context* (cf. Figure 4.2). Each representation is used to carrying out a concatenation between the dialogue history and the current utterance. The final representation starts with a [*CLS*] token followed by the dialogue representation, then the [*SEP*] token is used for delimiting the representation (cf. Figure 4.1).

4.2.2 Feature Extraction Module

As similar to the procedure presented in Section 3.2.1, the feature extraction module is based on BERT [18]. Figure 4.1 presents the user utterance and system response from the previous turn as dialogue context (purple circle, C). Our solution also uses the current user utterance message as input, represented as a token sequence in BERT input format. The first token is [*CLS*], followed by the tokenized user utterance and system response, then the current tokenized user utterance, and [*SEP*]. For instance, in Figure 4.1 we use the following token sequence. ‘[*CLS*] [*USER*] *consultar pontuação mundo* [*SYSTEM*] *consultando pontos* [*USER*] *credito boleto faço ... desconto* [*SEP*]’.

Formally, let $[x_0, x_1, \dots, x_n]$ denoting the input token sequence, then BERT input layer embeds each token x_i into an embedding e_i , which is the sum of three embeddings (input defined in Section 2.1.3.4):

$$BERTinput(x_i) = Etok(x_i) + Eseg(i) + Epos(i), e_i \in \mathbb{R}^d, \forall 0 \leq i \leq n \quad (4.1)$$

where $Etok$ is the token embedding, $Eseg$ is the segment embeddings, and $Epos$ is the position embedding for each token x_i . The embedded input sequence $[e_0, \dots, e_n]$ is then passed to BERT bidirectional Transformer encoder, whose final hidden states are denoted by $[b_0, \dots, b_n]$ (cf. Figure 4.1).

$$[b_0, \dots, b_n] = BiTransformer([e_0, \dots, e_n]), w_i \in \mathbb{R}^d, \forall 0 \leq i \leq n \quad (4.2)$$

Our solution uses a contextualized sentence-level representation output (b_0) as a final state corresponding to the [*CLS*] token.

4.2.3 Classification Module

The input for the classification module is the sentence-level representation b_0 from the feature extraction module. The classification module predicts the value of b_0 to be one of the twenty-two categories using a convolutional neural network and softmax.

We use a CNN to capture intrinsic syntactic and semantic patterns from input sentences [64]. CNN consists of convolution, pooling, and activation layers [47]. At each

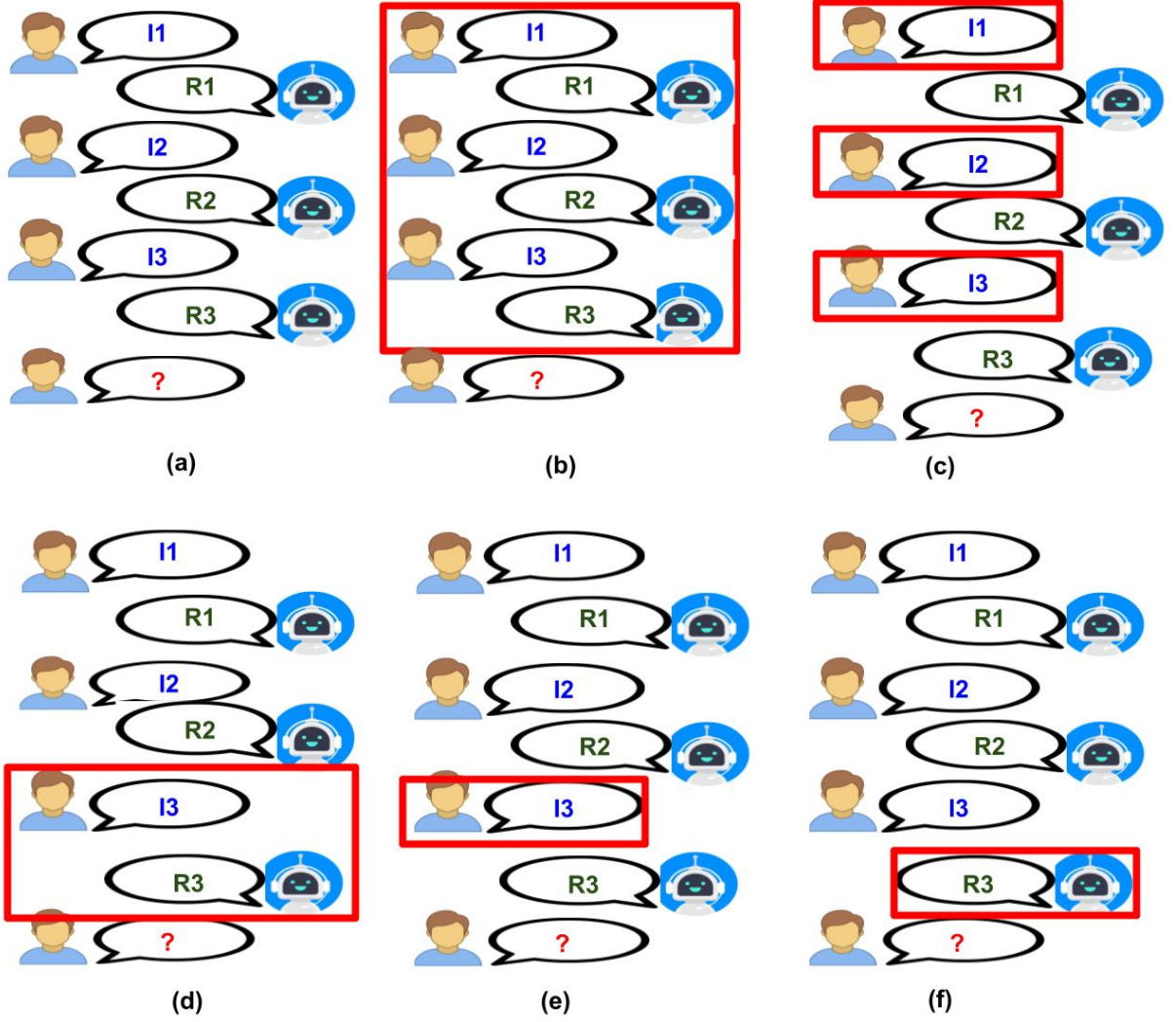


Figure 4.2: Several types of trajectories (from left to right) in our window-based approach: (a) shows a communication process between a user (left) and a bot (right) representing a complete dialog flow, where I_i (blue message) corresponds to i th utterance from the user, and R_j (green message) corresponds to a j th response from a bot. We visualize the current utterance without associated intent as a question mark '?' (red message); (b) exhibits an approach called '*all context*' that uses the complete previous information (user and bot) as context; Similarly, (c) presents a window-based approach called '*user context*' that employs the user's previous messages to get the context; (d) reveals a window-based approach denominated '*user-bot context*' that employs the last utterance from user and from the bot to get the context; (e) reveals a window-based approach denominated '*last-user context*' that employs the user's last utterance to get the context. Finally, (f) exposes a window-based approach denominated '*last-bot context*' that employs the bot's last utterance to get the context.

convolution layer, a set of kernels convolved a vector, which acts as filters. The pooling layer is composed of a max-pooling function, which reduces the vector size and increases the receptive field size. The activation layer adds nonlinearity to the neural network. In this case, it is usually a Rectified Linear Units (ReLU) that replaces negative inputs with

0 and keeps the positive inputs unchanged [56].

The mini-batch gradient descent algorithm with RMSprop optimizer [33] was used to train the CNN parameters, with an initial learning rate of 0.1 (as [33] suggests). Batch size is set to 32 due to memory limitation. To reduce overfitting and improve generalization, we used the dropout technique [69] with dropout rates between 0.6 and 0.25. Also, our solution explored batch normalization [37] during training (to reduce the overfitting). To compute the error during training stage we used cross-entropy loss [71], we used this loss function because we compared several loss functions, and the most accurate results correspond to cross-used.

In our configuration evaluation in the training processing, we identified the best configuration to CNN architecture (cf. Table 4.1).

Table 4.1: Convolutional Neural Network configuration.

Type	Description	#filters	filter size	stride	#units	rate
BN	Batch normalization	-	-	-	-	-
conv	1D Convolution	64	(1×9)	1	-	-
Pool	Max-pooling	-	(1×2)	2	-	-
bn	Batch normalization	-	-	-	-	-
do	Dropout	-	-	-	-	0.6
conv	1D Convolution	32	(1×9)	1	-	-
Pool	Max-pooling	-	(1×2)	2	-	-
bn	Batch normalization	-	-	-	-	-
do	Dropout	-	-	-	-	0.6
FC	Fully-connected	-	-	-	30	-
bn	Batch normalization	-	-	-	-	-
do	Dropout	-	-	-	-	0.25
FC	Fully-connected	-	-	-	22	-

4.3 Experiments

We evaluated our models using several window-based approaches addressed in our work on the WvGD dataset.

4.3.1 Dataset

Section 3.3.1 described the features and challenges of the WvGD dataset. The full version presents 357.182 utterances, and the first cleaning created by the Wavy Global enterprise had 44.520 isolated utterances. For our current experiments, this variant was not adequate because we needed dialog flows. To this end, we built a new version from the complete WvGD version. Table 4.2 presents the description of the data columns in the dataset.

During our experiments, we do not use ATIS [31] because this dataset does not have dialogue flows.

Table 4.2: Dataset column detail.

Type	Description
Query	Text sent by user
Intent	Name of intent detected by the bot given the user’s query
Confidence	Bot’s confidence to detect the intent, from 0 to 1.
Response	Text sent by the bot as a reply to the user’s query.
Time	Date and time when the query was sent, in the format ‘2019-11-07 15:14:20 UTC’.
Channel	Whether the conversation took place on Facebook or Whatsapp.
User	An ID that identifies the user.
Session	An ID that identifies the session. A session is a conversation between the user and the bot, which lasts at most 24 hours. After that period, if the same user continues talking, another session is opened.

We refined the new version of the dataset by removing existing dialog flow sessions according to the following criteria. Table 4.3 shows summary statistics for the built corpus:

- Conversations where the user sent less than three messages in the same session because the dialogue flow would be short;
- Conversations where the user sent more than fifteen messages in the same session because the dialogue flow would be enormous;
- Conversations where the intent was ‘undefined’ because a lot of utterances were miscategorized. Dialogflow has assigned this value when the query does not belong to one of the categories;
- Conversations where the confidence value is lower than 0.70 to guarantee a safe level of effectiveness;
- Queries that are too long (> 140 characters) because they do not manifest relevant information. For instance, we find chain letters of whatsapp;
- Queries without letters or numbers (only symbols or emojis) because we did not define special tokens for each emoji.

Table 4.3: Corpus statistics for the new WvGD dataset.

Average number of words per query		3.75
Standard deviation per query		2.92
Variance per query		8.57
Average number of words per response		33.76
Standard deviation per response		30.65
Variance per response		939.48

The final distribution in the dataset was composed of dialog flows between three and fifteen user queries. Figure 4.3 shows the number of conversations and distribution for each dialog flow size.

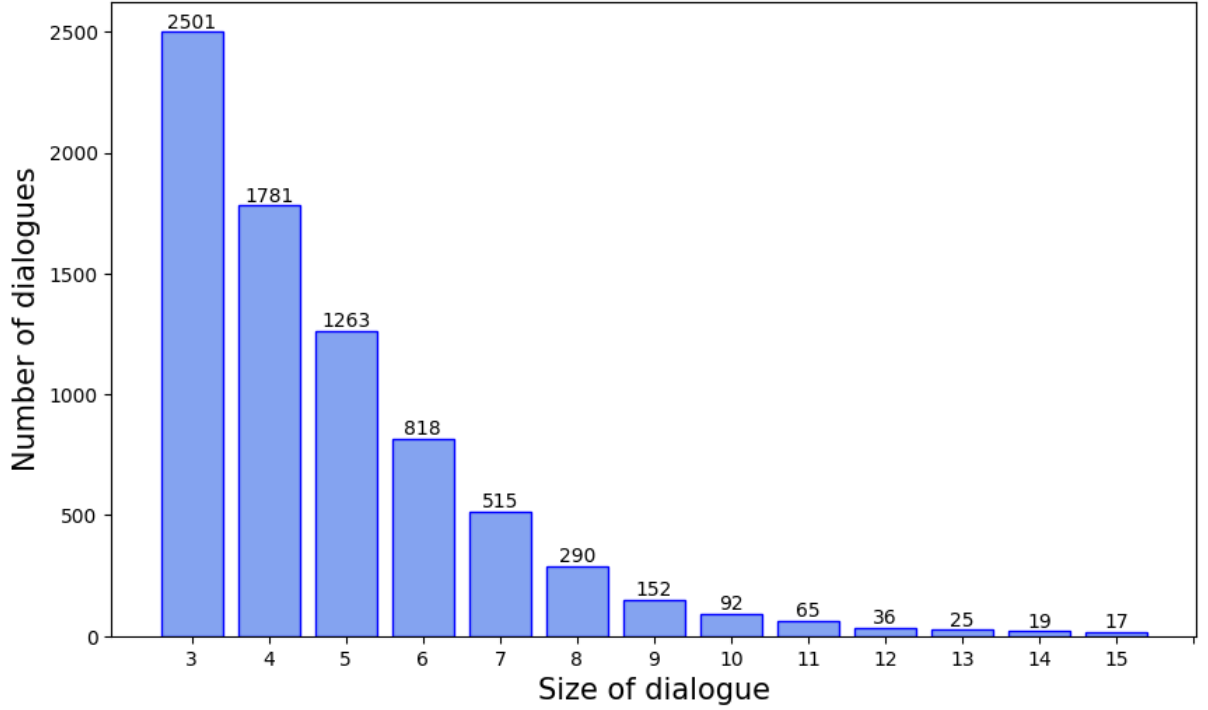


Figure 4.3: Distribution of the number of conversations for each dialogue flow size.

As far as the full number of utterances is concerned, there are 36.056 queries. Figure 4.4 shows the frequency and distribution of each intent class in the dataset.

4.3.2 Experimental Settings

In our experimental procedure, we randomly divided the WvGD into 60% of samples for training the classifier, 20% for validation, and 20% for testing it. We used the pre-trained [BERT-Base, Uncased] model, which has 12 hidden layers of 768 units and 12 self-attention

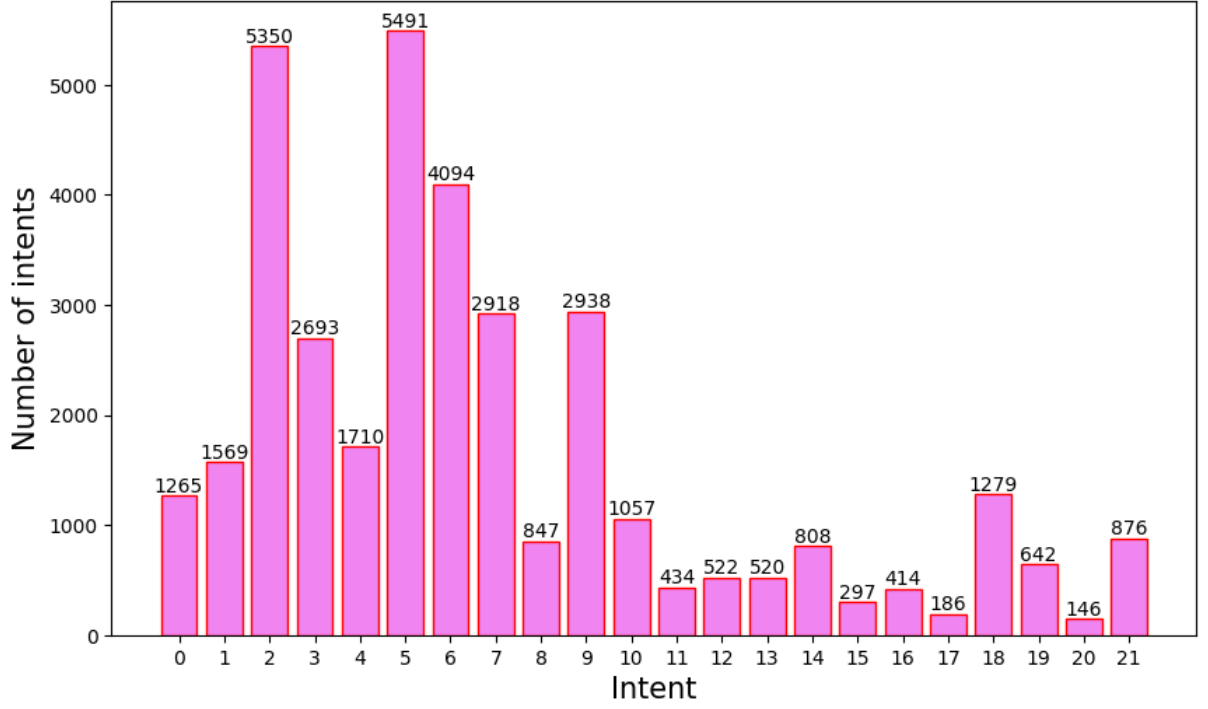


Figure 4.4: Distribution of number of intent. At the top of each bar we can see the exact amount of intents. Where 0, 1, ... , 21 correspond to the class labels, for example: initial_greetings , [CT]bank_slip or [IT]delivery_problem. For confidentiality reasons, we cannot reveal more information about the labels

heads for lower-cased input text. We used their feature extraction effectiveness to apply our window-based approach.

About the convolutional neural network architecture configuration, the loss function used was a cross-entropy loss for the corresponding prediction target. We updated all layers in the model using RMSprop optimization [33] and early stopping on the validation set. During training, we used 25% and 60% dropout rates, also batch normalization. Finally, the model was trained for 50 epochs.

First, we run one baseline without context based on our BERT solution. We contrast the results from this baseline against our windows-based approaches proposed (all context; user context; user-bot context; last-user context; and last-bot context). In this chapter, we conducted experiments on the new WvGD dataset version. We reported the evaluation effectiveness of our models for intent classification based on the counts of test records correctly and incorrectly predicted by the proposed models. In this context, five types of metrics were used to measure the prediction system correctness, as follows: confusion matrix, accuracy, recall, precision, and f1-score. Section 4.4 reports on the obtained results.

4.4 Results and Discussion

Overall results. Tables 4.4 and 4.5 present the comparison between our models and the baseline model on the validation and test sets, respectively.

Table 4.4: Comparison between isolated intent classification (without context) and intent classification based on window-based approaches for the **WvGD validation set**. The highest results are highlighted in red. Baseline results are highlighted in blue.

Approach	Accuracy	Recall	Precision	F1-Score
without context	85.56	83.26	87.71	85.37
all context	70.09	59.40	86.48	70.02
user context	77.31	68.19	88.58	76.65
user-bot context	85.33	82.07	90.46	85.98
last-user context	87.61	85.77	90.10	87.84
last-bot context	86.06	83.93	90.02	86.81

Table 4.5: Comparison between isolated intent classification (without context) and intent classification based on window-based approaches for the **WvGD test set**. The highest results are highlighted in red. Baseline results are highlighted in blue.

Approach	Accuracy	Recall	Precision	F1-Score
without context	85.32	83.15	87.55	85.25
all context	66.52	55.22	80.54	65.14
user context	78.00	69.34	88.91	77.57
user-bot context	86.08	82.94	90.55	86.49
last-user context	87.18	85.01	89.79	87.28
last-bot context	86.56	84.18	89.85	86.88

Results in Tables 4.4 and 4.5 reveal that our approaches based on the last utterance from user and bot (*user-bot context*, *last-user context*, and *last-bot context*) outperformed the baseline model in terms of accuracy, recall, precision, and f1-score. In contrast, baseline model presented better results for two approaches based on all conversation, *all context* and *user context*. A plausible explanation for the significant higher score of the approaches based on user and bot last utterances is that our technique uses recent dialogue history, and the baseline does not. We found that the use of full information from the context does not contribute for the classification as relevant knowledge to current utterance. Conversely, it adds unnecessary information, for instance, *greetings*. These

results make sense because a dialogue is not just a sequence of independent utterances, but rather a collective action performed by the speaker and the system [38].

The '*all context*' approach exhibited the worst effectiveness in terms of accuracy, recall, precision, and f1 score. A potential explanation is that the model saw no relevant information (noise) during training. BERT can accept 512 tokens as input (we used only 510 tokens because two tokens correspond to *[CLS]* and *[SEP]*). We employed all tokens for user utterance and 30 tokens for system response.

Figure 4.3 showed that the distribution and frequencies of dialogues varies between 3 and 15 queries. Suppose we have a dialog flow with twelve queries. First, we have to confirm the x tokens for the current utterance, so now we have $510 - x$ tokens. Second, we have eight tokens on average for each user utterance, 96 user tokens (8×12) and 360 system replies (30×12). Adding these tokens, we have 456 tokens. Therefore, we use 456 tokens as dialogue history. Consequently, if the dialogue has several queries, the model uses much information, which is confused with noise.

Confusion matrix results. Figures 4.5, 4.6, 4.7, 4.8, 4.9 and 4.10 present the results in terms of confusion matrix. The diagonal elements show the number of correct classifications for each class.

Figure 4.5 (results without context) presents the need of previous information, because in intent with label equal to '0' has main diagonal value equals to 7. The user's ID is essential to explain this results. In Table 4.6 shows that when the query is equal to '2345789', the model release two intent, labels equal to '0' and '2'. In the dataset, we saw intents with labels equal to '0' when the user started the conversation with the ID. For instance, the left part of Table 4.6, 'Without context'. However, if there is previous information, for example, 'Greetings', the intent will have a label equal to '2' (right part of Table 4.6). Nevertheless, the results without context only used the current utterance. Therefore, this approach assumes that labels '0' and '2' are the same. For this reason, there are so many false negatives when trying to classify the label equal to '0'.

Table 4.6: Importance of context during intent classification task. We show two parts, the left part, an utterance without context (red letters), and the right part, an utterance with context (blue letters). In both parts, the query is the same, '2345789' (green letters). However, the intent predicted is different. These results happen because there is a previous dialogue.

Without context				With context			
Turn	Actor	Query	Intent	Turn	Actor	Query	Intent
T	User	2345789	0	T	User	Bom dia	5
					System	olá, digite registro cadastral	
				T+1	User	2345789	2

By contrasting the results in Figure 4.5 with Figures 4.6 and 4.7, we observe several differences. Most important, the main diagonal in Figure 4.5 does not contain zeros. There are zero values in Figure 4.6 for intents with labels equal to '15', '17', and '20'.

Curiously, intent with label 7 receives much of false negatives from the classes mentioned before, i.e., predictions that should be labels equal to '15', '17' or '20' our model classifies as '7'. This fact is because the number of queries to train our model is unbalanced. While intention with label '7' has more than 2.000 sentences, the sum of intentions with labels equal to '15', '17', and '20' is 629. In Table 4.7, we present that there is a significant semantic similarity between queries associated with the mentioned labels, '7', '15', '17', and '20'. Therefore, this imbalanced database brings these problems assumes that labels equal to '7', '15', '17', and '20' are equivalent. For this reason, there are so many false negatives when trying to classify the labels equal to 15', '17', and '20'.

Table 4.7: Queries in different sessions with a certain level of semantic similarity. Part of the query '**quero mudar** equipe' (green), the utterance '**consigo passar novo pedido**' (red) and the message '**novo modelo de negócio**' (orange) are quite similar semantically to the sentence of '**outros assuntos**' (blue). To summarize, these examples represent statement very similars.

Session	Turn	Actor	Query	Intent
124	T^1	User	outros assuntos	7
1225	T^2	User	quero mudar equipe	15
5112	T^3	User	consigo passar novo pedido	17
8763	T^4	User	novo modelo de negócio	20

Similarly, there are zero values in the main diagonal in Figure 4.7 for labels equal to '11' and '17'. The same problem arises when there is a limited number of samples to train the model. Specifically, there are 2.938 samples with labels equal to '9'. Meanwhile, there are 434 examples with labels equal to '11'. Similarly, we only have 186 utterances with labels equal to '17'. Therefore, this imbalanced database brings these problems assumes that sentences with labels equal to '9', '11' and '17' are equivalent. For this reason, there are so many false negatives when trying to classify the labels equal to '11' and '17'.

The best results corresponded to the approaches regarding the context *user-bot context*, *last-user context*, and *last-bot context* shown in Figures 4.8, 4.9 and 4.10, respectively.

We observe in Figure 4.8 that there are still values equivalent to zero in the main diagonal, specifically in intention with label equal to '17'. This limitation was explained in Table 4.7 (utterances with a semantic meaning very similar). On the other hand, Figure 4.10 corresponds to the first approach ('last-bot context') that presents results different from 0 in intention with label equal to '17'. Therefore, using recent information from the system helps to mitigate this problem.

Based on that idea, we use recent user information as context, Figure 4.9 presents improvements over previous approaches. This approach ('last-user context') validates importance of the most recent user's utterance. In summary, enriching contextual information using only current information does not help when working with dialogue flows.

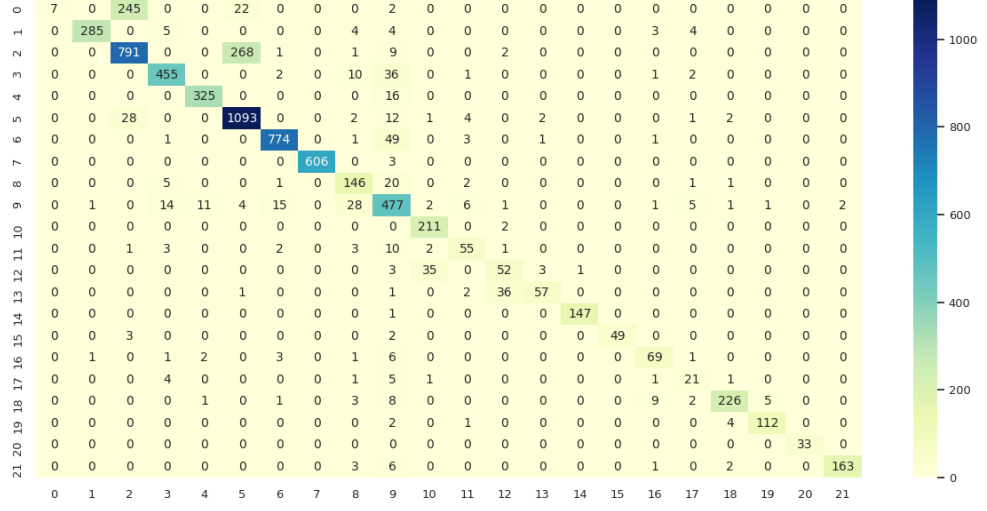


Figure 4.5: Confusion matrix regarding the results of intent classification without context on WvGD test dataset.

Dealing with imbalanced datasets. Obtained results do not consider dealing with imbalanced datasets, in which some of the classes appear much more often in the dataset than others (cf. Figure 4.4). The problem is that the model is likely to learn to predict only the dominant classes. Measures like confusion matrix provide an overview of data classified correctly and incorrectly. Also, Tables 4.4 and 4.5 showed very poor recall compared to results presented by other measures – precision and accuracy were higher than recall. In this context, the best thing we can do is to collect more data of the underrepresented classes. This is hard because we are working with real-world Brazilian Portuguese conversational data.

There are two approaches to solve this problem. First, the idea is to resample our dataset so it is better balanced [2]. Undersampling¹ and Oversampling² are the most common ways.

Undersampling is the easiest way by just randomly throw away samples from the dominant class. We can use some unsupervised clustering method and throw out only samples from the big clusters. The problem of course is that we are throwing out valuable data.

Oversampling is more complicated. We can repeat samples, but it is proved to not working very well [32]. We can also use SMOTE³ methods [9] (Synthetic Minority Over-sampling Technique) to generate new samples interpolating between existing ones although this may not be easy for complex utterances. A potential strategy is to modify

¹Undersampling throw away some examples of the dominant classes

²Oversampling get more samples of the underrepresented class

³Oversampling method that creating synthetic minority class examples, using Naive Bayes classifier [62]

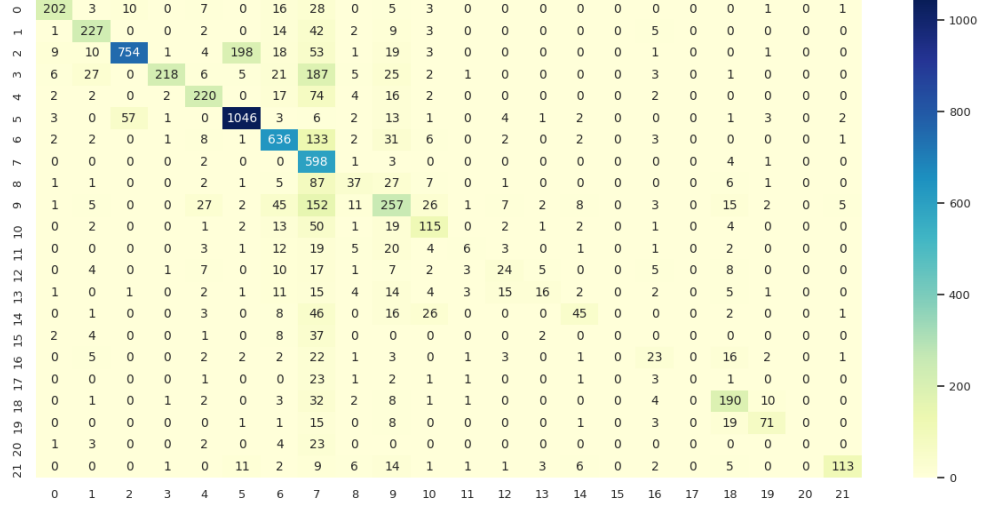


Figure 4.6: Confusion matrix regarding the results of intent classification using all context on WvGD test dataset.

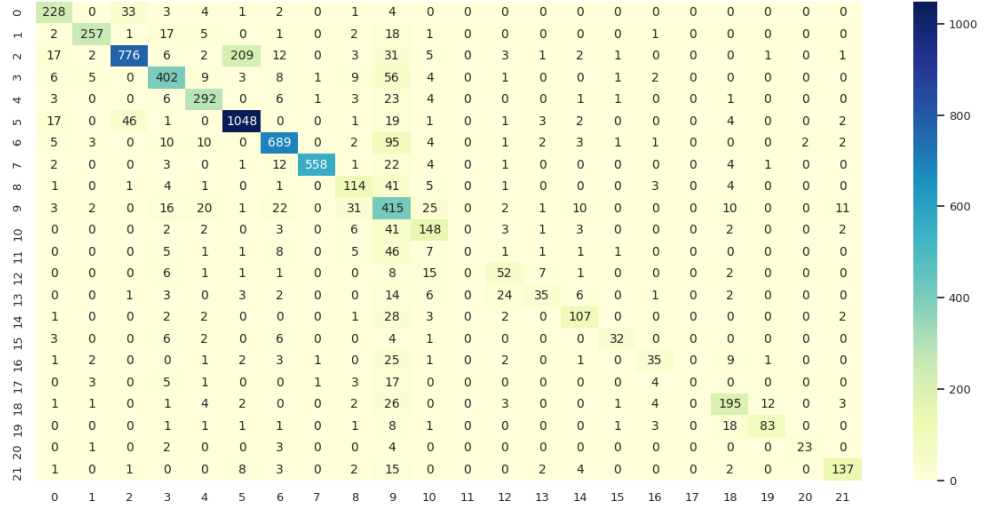


Figure 4.7: Confusion matrix regarding the results of intent classification using user context on WvGD test dataset.

our loss function to penalize misclassification of the underrepresented classes more than the dominant ones. We modify the cross-entropy loss function adding the loss value for each label (equation 4.3). Table 4.8 shows the loss values distribution for each class.

$$L_{CE} = - \sum_{i=1}^n t_i \log(P_i) \times L_i, \text{ for } n \text{ classes}, \quad (4.3)$$

where t_i is the truth label, P_i is the softmax probability and L_i is the loss value for

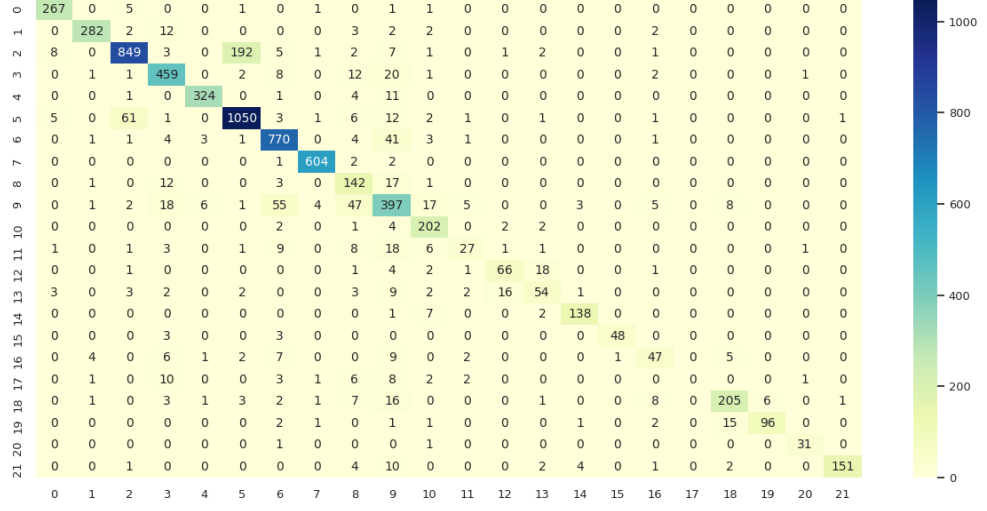


Figure 4.8: Confusion matrix regarding the results of intent classification using the last user-bot context on WvGD test dataset.

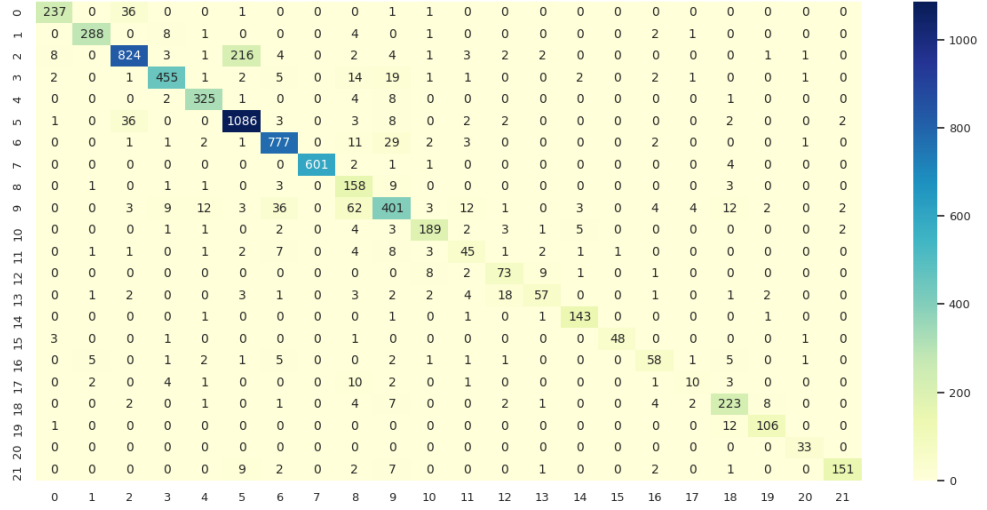


Figure 4.9: Confusion matrix regarding the results of intent classification using the last-user context on WvGD test set

the i^{th} class.

Using the modification of our loss value (we divide ten by the percentage of each label to obtain a specific loss value) during our model's training process as a strategy to address the imbalance of our database is essential because it increases the accuracy during the classification process and allows us to eliminate false positives. As a consequence, adding loss values implies that we need to re-train our model, tuning its penalization during training as reported in Table 4.8.

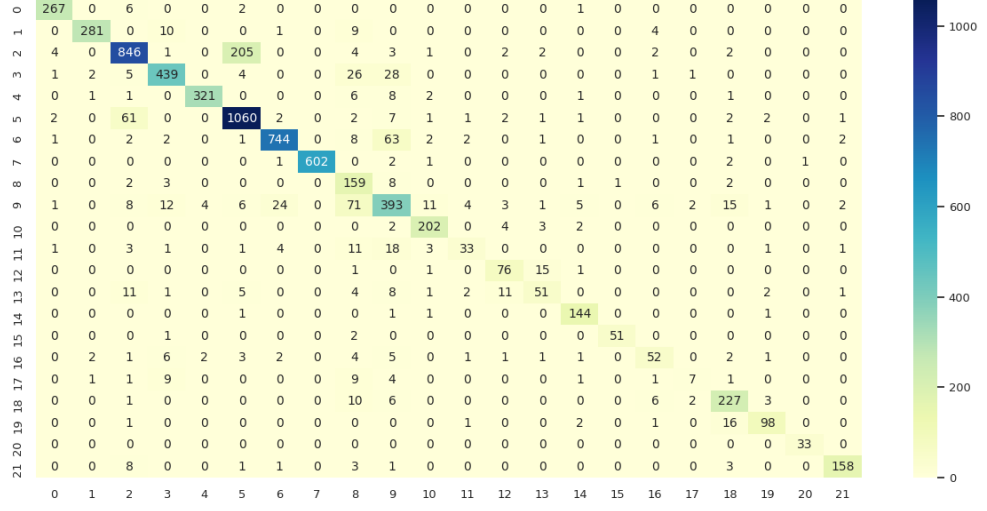


Figure 4.10: Confusion matrix regarding the results of intent classification using the last-bot context on WvGD test set.

Table 4.8: Loss function distribution for each intent class. The first column indicates the categories. The second column counts the occurrences of frequent intents. The third and fourth columns show each intent’s percentage and loss function computed.

Intent	# Intent	Percentage	Loss value	Intent	# Intent	Percentage	Loss value
0	1265	3.51	3.0	11	434	1.20	8.0
1	1569	4.35	2.0	12	522	1.45	7.0
2	5350	14.84	1.0	13	520	1.44	7.0
3	2693	7.47	1.0	14	808	2.24	4.0
4	1710	4.74	2.0	15	297	0.82	12.0
5	5491	15.23	1.0	16	414	1.15	9.0
6	4094	11.35	1.0	17	186	0.52	19.0
7	2918	8.09	1.0	18	1279	3.55	3.0
8	847	2.35	4.0	19	642	1.78	6.0
9	2938	8.15	1.0	20	146	0.40	25.0
10	1057	2.93	3.0	21	876	2.43	4.0

Tables 4.9 and 4.10 present the new results adapting our loss function representing validation and test sets, respectively.

Tables 4.9 and 4.10 present the results modifying our loss function on our window-based approaches. Our current model gets better results using the last utterance from the user, and bot (*user-bot context*, *last-user context*, and *last-bot context*). In the same way, using the *last-user context* approach as dialogue history contributes the best context to predict the current user’s utterance.

We found that adding a loss function in the training process allows mitigating the limitations presented in our previous results (cf. confusion matrices in Figures 4.5, 4.6, 4.7, 4.8, 4.9 and 4.10).

Table 4.9: Comparison between isolated intent (without context) and windows-based approaches for intent classification in the WvGD validation dataset using loss values. The highest results are highlighted in red. baseline results are highlighted in blue.

Approach	Accuracy	Recall	Precision	F1-Score
without context	86.00	85.44	87.02	86.21
all context	70.44	65.65	80.91	72.29
user context	78.63	70.34	88.93	78.22
user-bot context	85.65	83.12	89.30	86.05
last-user context	86.78	84.97	89.27	87.04
last-bot context	87.04	85.21	89.76	87.39

Table 4.10: Comparison between isolated intent (without context) and windows-based approaches for intent classification in the WvGD test set using loss values. The highest results are highlighted in red. Baseline results are highlighted in blue.

Approach	Accuracy	Recall	Precision	F1-Score
without context	85.19	84.62	86.17	85.37
all context	70.77	66.85	79.25	72.38
user context	78.64	71.55	88.67	78.92
user-bot context	86.21	83.97	90.14	86.89
last-user context	87.58	85.96	89.49	87.65
last-bot context	87.45	85.90	89.37	87.57

Novel results with the adding of the loss function (cf. confusion matrices in Figures 4.11, 4.12, 4.13, 4.14, 4.15 and 4.16) reveal that only the *user context* approach presents values equivalent to label '0' in its corresponding confusion matrix (Figure 4.13). This demonstrates that this approach is not recommended as a dialogue history.

Similarly, *all context* approach is not suggested because it has limited results. Nevertheless, labels equivalent to '0' in the confusion matrix were deleted.

We recommend using the last three approaches (*user-bot context*, *last-user context*, and *last-bot context*) to mitigate the intentions equivalent to 0 in the confusion matrix. These approaches showed the highest results compared to the baseline.

4.5 Conclusion

Intent classification in dialog flow processing can benefit from addressing contextual information. In this work, we proposed windows-based approaches for intent classification tasks. Our window-based strategy made use of distinct types of windows on the con-

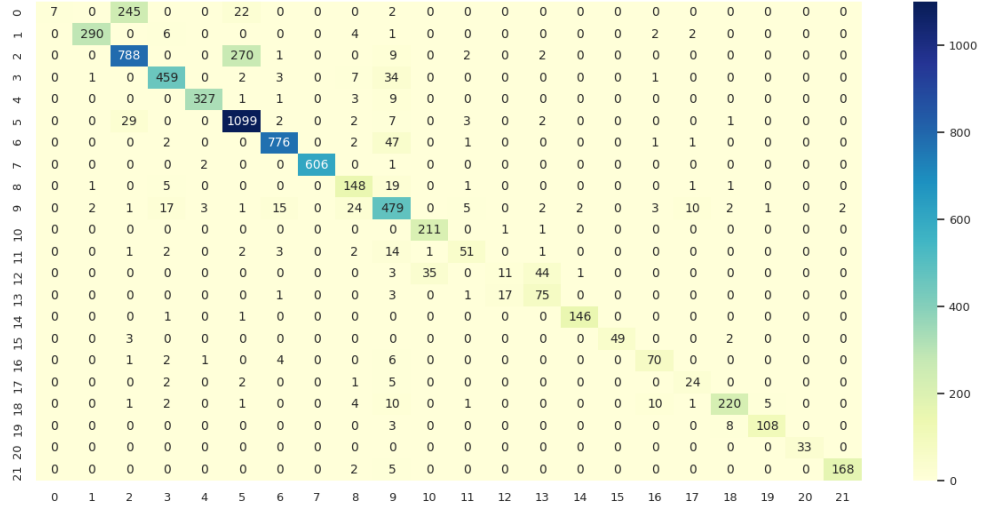


Figure 4.11: Confusion matrix regarding the results of intent classification with new loss function without context on WvGD test dataset.

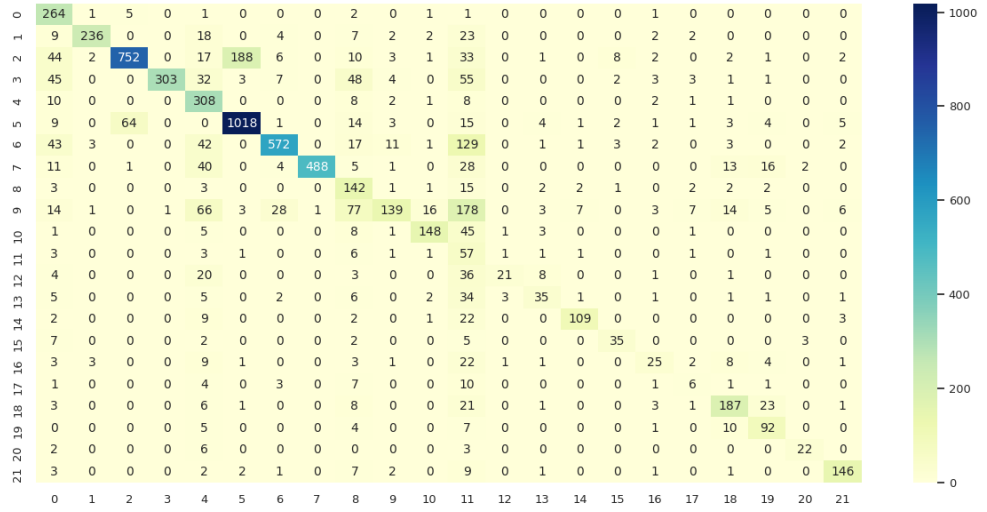


Figure 4.12: Confusion matrix regarding the results of intent classification with new loss function using all context on WvGD test dataset.

textual information in the dialog flow sessions. We identified and defined five types of trajectories. Our study used and experimented these types of trajectories as the contextual information. For experimental purpose, we created a new version of WvGD dataset, which provided dialogue flows of various sizes between 3 and 15 queries. We conducted experiments to understand the added value of our defined windows-based approaches for intent classification. Our results achieved substantial improvements over the baseline in three approaches using the user utterance and system response from previous messages

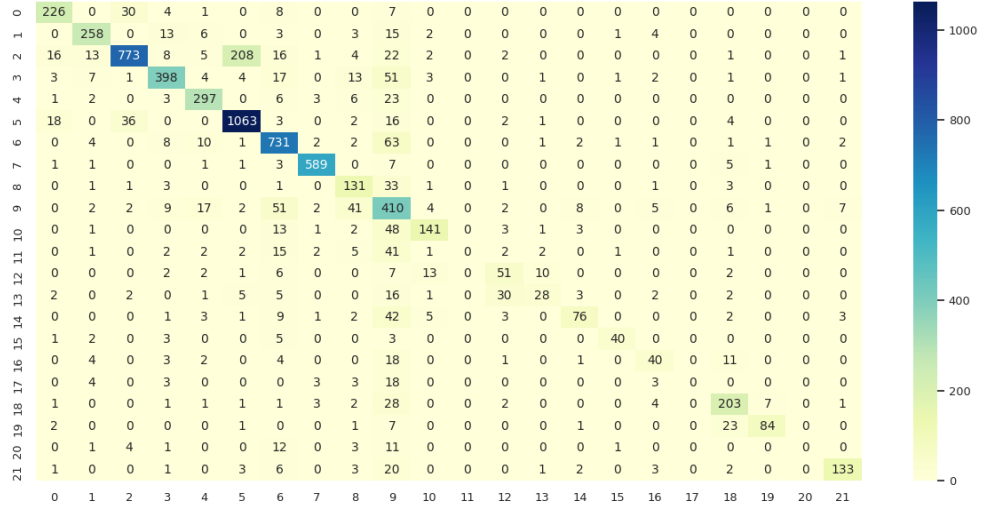


Figure 4.13: Confusion matrix regarding the results of intent classification with new loss function using user context on WvGD test dataset.

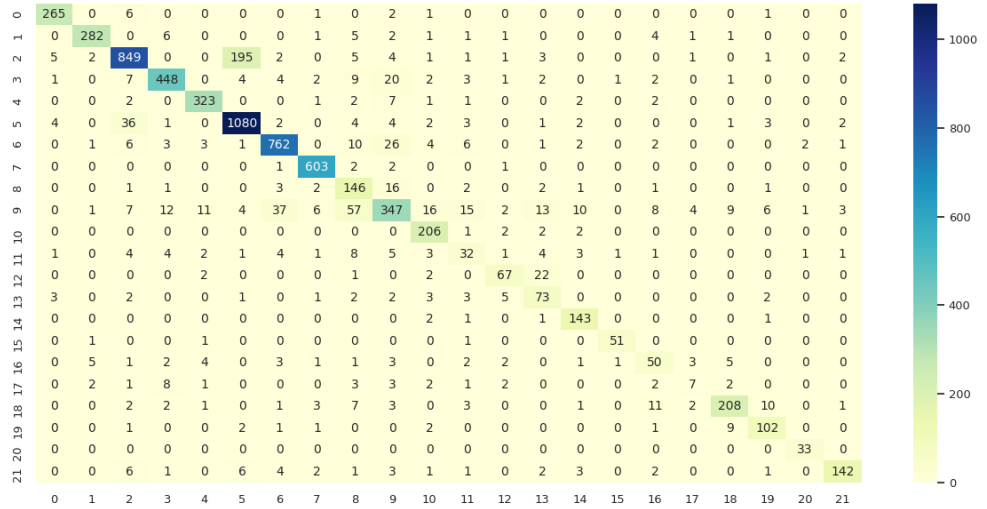


Figure 4.14: Confusion matrix regarding the results of intent classification with new loss function using the last user-bot context on WvGD test dataset.

as dialogue context. In addition, for addressing unbalanced aspects and refine the results, we investigated and explored modification in our loss value in the model training process. This enabled us to address the imbalance of our dataset. Final results showed benefits in this decision and increased the accuracy during the classification process. It allowed us to eliminate false positives as showed our results in the matrices of confusion.

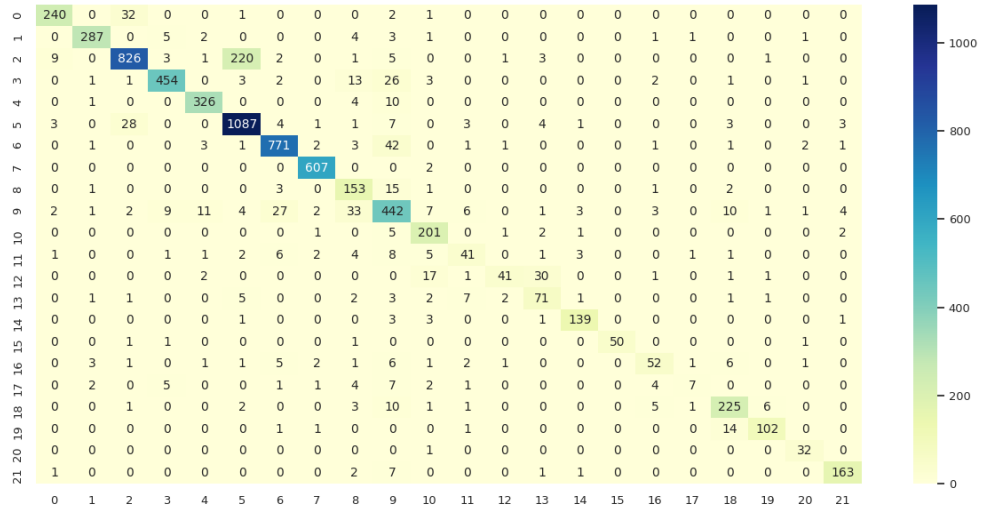


Figure 4.15: Confusion matrix regarding the results of intent classification with new loss function using the last-user context on WvGD test dataset.

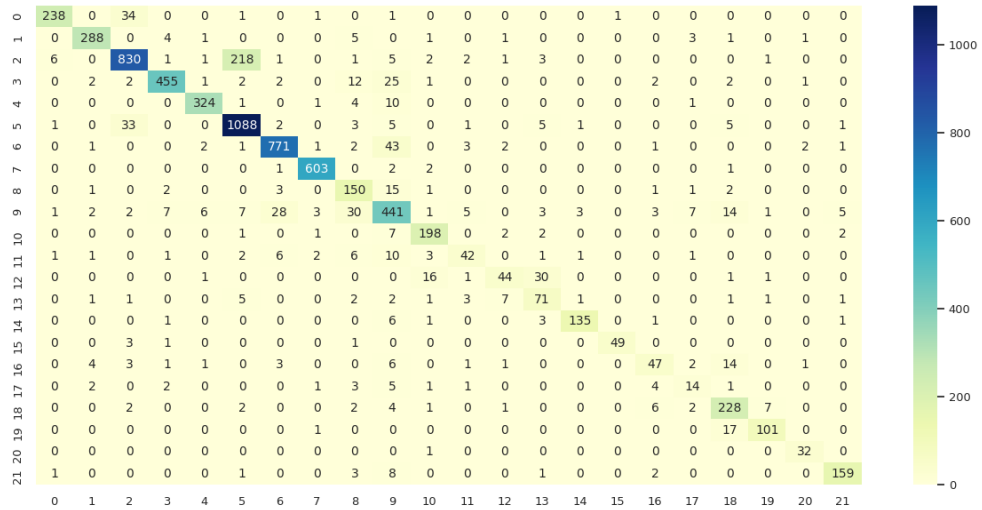


Figure 4.16: Confusion matrix regarding the results of intent classification with new loss function using the last-bot context on WvGD test dataset.

Chapter 5

Conclusion

Intent classification in natural language messages is a challenging task. An utterance can contain outliers, grammatical errors, and many categories. Also, from the literature review, an additional obstacle corresponds to the fact that there are no studies for the Brazilian Portuguese language in this task. In Chapter 3, we studied several vector representations for isolated utterances in the Brazilian Portuguese language. We compare various classical embeddings (Word2Vec [55], Glove [58] and FastText [3]) and a pre-trained model (BERT [18]) with the aim of identifying the best representation for a utterance. In the same line, we evaluated and compared the effectiveness of different classifiers, namely Support Vector Machines (SVM), Logistic Regression (LR), Neural Network (NN), and a Convolutional Neural Network (CNN). Finally, we concluded that pre-trained models like BERT are more adequate than classical word embeddings for message representation in the investigated task. Correspondingly, we improved our results training a CNN architecture. Moreover, we found that the analyzed active learning techniques minimized the amount of labeled data required to build accurate prediction models and improved accuracy compared to the supervised learning approaches.

After identifying the best embedding, in Chapter 4, we address context (one or a few previous utterances) to guide the process of learning features that identify user's intent in a dialog flow. We proposed windows-based approaches for intent classification tasks. Our strategy used distinct types of windows on the contextual information in the dialog flow session. Also, we created a new version of WvGD dataset for experimental purposes, which presents dialogue flows of various sizes between 3 and 15 queries. We contrast our approach with a baseline, a version without context. Our results demonstrated that using user utterance from the previous turn as dialogue context achieves the best results. Also, we investigated and explored modification in our loss value in the model training process for addressing imbalanced classes aspects and refining the results. Wavy Global enterprise validated our models through another private dataset, getting outperform results during the testing process.

According Liu *et al.* [49] none of the publicly available Natural Language Understanding (NLU) toolkits, such as, Dialogflow, LUIS and Rasa, use dialogue context for the intent

classification task. However, we presented a windows-based approach that demonstrates our method’s effectiveness using context during intent classification task that obtains accurate results in a challenging dataset.

To summarize, we compared several embeddings and concluded that pre-trained models like BERT are more adequate to represent isolated utterances in the Brazilian Portuguese language. Then, to capture intrinsic syntactic and semantic patterns, from the results, we assumed that training a convolutional neural network architecture is the best option. Finally, we proposed numerous trajectories of our windows-based approach to investigate the relevance of context. Our experiments demonstrated that the context must be the user utterance from the previous turn to outperform the baseline result.

Based on the literature review, previous works addressed intent classification task using only isolated utterances and disconnected intentions within a dialogue flow. Conversely, Dialogue State Tracking and dialogue acts tasks combine the semantics extracted by the natural language understanding module with the previous dialogue context to estimate the current dialogue state. Inspired by this scheme, our windows-based approach uses a dialogue history and the current utterance to improve the natural language understanding module. Especially, intent classification task.

Based on the conducted experiments, the hypothesis that results during intent classification task can be enhanced by the use of a Deep architecture and previous information for the problem has been validated in our work.

In the following, we present the summary of contributions obtained in the M.Sc. dissertation, as well as point out possible research avenues for future investigations.

5.1 Summary of the Contributions

The main contributions of our investigation for intent classification are:

- A novel comparative study concerning several embeddings for intent classification in isolated utterances in Portuguese language.
- A novel comparative study concerning several classifiers for intent classification in isolated utterances in Portuguese language.
- Definition of an original approach for considering the context of utterances for automatic intent classification.
- Build of a cleaned dataset with dialogue flow to train new models.
- Extensive experiments results demonstrating the value of our proposal with the use of a real-world Brazilian Portuguese conversational dataset.
- An exhaustive analysis from measures. Mainly, the confusion matrix to add a loss function penalization based on an imbalanced dataset.

5.2 Future Perspectives

As directions for future work we propose the following:

- Study and propose refined techniques for active learning, other CNN architectures used in computer vision including Inception [72], Resnet [29], and Xception [12]. Besides, the evaluation of other pre-trained models to message representation such as Wang2vec [28]..
- Compare the results obtained with a model that eliminates the preprocessing module. Because current pre-trained models use all the information.
- Investigate the use of our windows-based approach on Dialogue State Tracking task because current works do not standardize the dialogue history. For instance, [77] and [26] used all the context, and [8] used the last system reply.
- Propose and experiment other approaches to intent classification relying on contextual information. For instance, using percentages of dialogue history to compute the relevance of context, or create a classifier to context pre-selection.

5.3 Dissemination of Results

Results generated in the studies of this M.Sc. dissertation was partially published in an international conference article as follows:

- ESCOBEDO, J. D. F., LOPES, K.; DOS REIS, J. C. 2021. *Active Learning Approach for Intent Classification in Portuguese Language Conversation*. In Proceedings of the IEEE XV International Conference on Semantic Computing (ICSC). Laguna Hills, CA, USA, pp. 227-232, DOI: 10.1109/ICSC50631.2021.00048.

Bibliography

- [1] J. Alammar. The illustrated transformer. <http://jalammar.github.io/illustrated-transformer/>. Accessed: 2019-08-23.
- [2] M. Bach, A. Werner, and M. Palt. The proposal of undersampling method for learning from imbalanced datasets. *Procedia Computer Science*, 159:125–134, 2019.
- [3] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov. Enriching word vectors with subword information. *Transactions of the Association for Computational Linguistics*, 5:135–146, 2017.
- [4] L. Breiman. Random forests. *Machine learning*, 45(1):5–32, 2001.
- [5] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*, 2020.
- [6] C. Campbell, N. Cristianini, A. Smola, et al. Query learning with large margin classifiers. In *ICML*, volume 20, page 0, 2000.
- [7] T. N. Cardoso, R. M. Silva, S. Canuto, M. M. Moro, and M. A. Gonçalves. Ranked batch-mode active learning. *Information Sciences*, 379:313–337, 2017.
- [8] G.-L. Chao and I. Lane. Bert-dst: Scalable end-to-end dialogue state tracking with bidirectional encoder representations from transformer. *arXiv preprint arXiv:1907.03040*, 2019.
- [9] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer. Smote: synthetic minority over-sampling technique. *Journal of artificial intelligence research*, 16:321–357, 2002.
- [10] Q. Chen, Z. Zhuo, and W. Wang. Bert for joint intent classification and slot filling. *arXiv preprint arXiv:1902.10909*, 2019.
- [11] J. Cheng, L. Dong, and M. Lapata. Long short-term memory-networks for machine reading. *arXiv preprint arXiv:1601.06733*, 2016.
- [12] F. Chollet. Xception: Deep learning with depthwise separable convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1251–1258, 2017.

- [13] A. Chopra. 21 vital chatbot statistics. <https://outgrow.co/blog/vital-chatbot-statistics>. Accessed: 2019-08-21.
- [14] K. Clark, M.-T. Luong, Q. V. Le, and C. D. Manning. Electra: Pre-training text encoders as discriminators rather than generators. *arXiv preprint arXiv:2003.10555*, 2020.
- [15] D. Clayton. Wellactually, americans say customer service is better than ever. <https://about.americanexpress.com/press-release/wellactually-americans-say-customer-service-better-ever>. Accessed: 2019-08-21.
- [16] A. Debecker. 3 stats that show chatbots are here to stay. <https://venturebeat.com/2016/08/26/3-stats-that-show-chatbots-are-here-to-stay/>. Accessed: 2021-01-11.
- [17] J. Deriu, A. Rodrigo, A. Otegi, G. Echegoyen, S. Rosset, E. Agirre, and M. Cieliebak. Survey on evaluation methods for dialogue systems. *arXiv preprint arXiv:1905.04071*, 2019.
- [18] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
- [19] J. C. dos Reis, R. Bonacin, H. H. Hornung, and M. C. C. Baranauskas. Intentionicons: Participatory selection of emoticons for communication of intentions. *Computers in Human Behavior*, 85:146–162, 2018.
- [20] A. Fadhil. Beyond patient monitoring: Conversational agents role in telemedicine & healthcare support for home-living elderly individuals. *arXiv preprint arXiv:1803.06000*, 2018.
- [21] J. D. Farfan-Escobedo, K. Lopes, and J. C. Dos Reis. Active learning approach for intent classification in portuguese language conversations. In *2021 IEEE 15th International Conference on Semantic Computing (ICSC)*, pages 227–232. IEEE, 2021.
- [22] J. G. FitzGerald. Stil–simultaneous slot filling, translation, intent classification, and language identification: Initial results using mbart on multiatis++. *arXiv preprint arXiv:2010.00760*, 2020.
- [23] A. Gallo. The value of keeping the right customers. <https://hbr.org/2014/10/the-value-of-keeping-the-right-customers>. Accessed: 2019-08-21.
- [24] N. Gonthier, Y. Gousseau, and S. Ladjal. An analysis of the transfer learning of convolutional neural networks for artistic images. *arXiv preprint arXiv:2011.02727*, 2020.

- [25] A. K. Gopalakrishna, T. Ozcelebi, A. Liotta, and J. J. Lukkien. Relevance as a metric for evaluating machine learning algorithms. In *International Workshop on Machine Learning and Data Mining in Pattern Recognition*, pages 195–208. Springer, 2013.
- [26] P. Gulyaev, E. Elistratova, V. Konovalov, Y. Kuratov, L. Pugachev, and M. Burtsev. Goal-oriented multi-task bert-based dialogue state tracker. *arXiv preprint arXiv:2002.02450*, 2020.
- [27] M. Hagiwara. Real world natural language processing. *Practical Applications with Deep Learning*, 2020.
- [28] N. Hartmann, E. Fonseca, C. Shulby, M. Treviso, J. Rodrigues, and S. Aluisio. Portuguese word embeddings: Evaluating on word analogies and natural language tasks. *arXiv preprint arXiv:1708.06025*, 2017.
- [29] K. He, X. Zhang, S. Ren, and J. Sun. Deep Residual Learning for Image Recognition. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.
- [30] M. A. Hearst, S. T. Dumais, E. Osuna, J. Platt, and B. Scholkopf. Support vector machines. *IEEE Intelligent Systems and their applications*, 13(4):18–28, 1998.
- [31] C. T. Hemphill, J. J. Godfrey, and G. R. Doddington. The atis spoken language systems pilot corpus. In *Speech and Natural Language: Proceedings of a Workshop Held at Hidden Valley, Pennsylvania, June 24-27, 1990*, 1990.
- [32] J. Hernandez, J. A. Carrasco-Ochoa, and J. F. Martínez-Trinidad. An empirical study of oversampling and undersampling for instance selection methods on imbalance datasets. In *Iberoamerican Congress on Pattern Recognition*, pages 262–269. Springer, 2013.
- [33] G. Hinton, N. Srivastava, and K. Swersky. Neural networks for machine learning lecture 6a overview of mini-batch gradient descent. *Cited on*, 14(8), 2012.
- [34] K. Hornik, M. Stinchcombe, and H. White. Multilayer feedforward networks are universal approximators. *Neural networks*, 2(5):359–366, 1989.
- [35] J. Howard and S. Ruder. Universal language model fine-tuning for text classification. *arXiv preprint arXiv:1801.06146*, 2018.
- [36] R. Hu, S. J. Delany, and B. Mac Namee. Egal: Exploration guided active learning for tcbr. In *International Conference on Case-Based Reasoning*, pages 156–170. Springer, 2010.
- [37] S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456. PMLR, 2015.

- [38] D. Jurafsky and J. H. Martin. Speech and language processing (draft). *Chapter A: Dialog Systems and Chatbots (Draft of October 16, 2019)*, 19:2019, 2019.
- [39] Å. Kamphaug, O.-C. Granmo, M. Goodwin, and V. I. Zadorozhny. Towards open domain chatbots—a gru architecture for data driven conversations. In *International Conference on Internet Science*, pages 213–222. Springer, 2017.
- [40] A. Kantarci. 84 chatbot /conversational ai statistics: Market size, adoption. <https://research.aimultiple.com/chatbot-stats/>. Accessed: 2021-01-11.
- [41] H. Khanpour, N. Guntakandla, and R. Nielsen. Dialogue act classification in domain-independent conversations using a deep recurrent neural network. In *Proceedings of COLING 2016, the 26th International Conference on Computational Linguistics: Technical Papers*, pages 2012–2021, 2016.
- [42] J. Kim, S. Y. Park, and L. P. Robert. Bridging the health disparity of african americans through conversational agents. *Digital Government: Research and Practice*, 2(1):1–7, 2020.
- [43] J. Kruk, J. Lubin, K. Sikka, X. Lin, D. Jurafsky, and A. Divakaran. Integrating text and image: determining multimodal document intent in instagram posts. *arXiv preprint arXiv:1904.09073*, 2019.
- [44] H. Kumar, A. Agarwal, R. Dasgupta, and S. Joshi. Dialogue act sequence labeling using hierarchical encoder with crf. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.
- [45] S. Larson, A. Mahendran, J. J. Peper, C. Clarke, A. Lee, P. Hill, J. K. Kummerfeld, K. Leach, M. A. Laurenzano, L. Tang, et al. An evaluation dataset for intent classification and out-of-scope prediction. *arXiv preprint arXiv:1909.02027*, 2019.
- [46] H. Le, D. Sahoo, C. Liu, N. F. Chen, and S. C. Hoi. Uniconv: A unified conversational neural architecture for multi-domain task-oriented dialogues. *arXiv preprint arXiv:2004.14307*, 2020.
- [47] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [48] J. Ledesma. Why a personalized consumer experience matters in the ecommerce world. <https://business.trustpilot.com/reviews/why-a-personalized-consumer-experience-matters-in-the-ecommerce-world>. Accessed: 2019-08-21.
- [49] X. Liu, A. Eshghi, P. Swietojanski, and V. Rieser. Benchmarking natural language understanding services for building conversational agents. *arXiv preprint arXiv:1903.05566*, 2019.

- [50] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.
- [51] J. Lu, M. Henchion, and B. Mac Namee. Investigating the effectiveness of word-embedding based active learning for labelling text datasets. *arXiv preprint arXiv:1910.03505*, 2019.
- [52] L. Meng and M. Huang. Dialogue intent classification with long short-term memory networks. In *National CCF Conference on Natural Language Processing and Chinese Computing*, pages 42–50. Springer, 2017.
- [53] L. Meng and M. Huang. Dialogue intent classification with long short-term memory networks. In X. Huang, J. Jiang, D. Zhao, Y. Feng, and Y. Hong, editors, *Natural Language Processing and Chinese Computing*, pages 42–50, Cham, 2018. Springer International Publishing.
- [54] I. V. Meza, L. Salinas, E. Venegas, H. Castellanos, A. Chavarría, and L. A. Pineda. Specification and evaluation of a spanish conversational system using dialogue models. In *Ibero-American Conference on Artificial Intelligence*, pages 346–355. Springer, 2010.
- [55] T. Mikolov, K. Chen, G. Corrado, and J. Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.
- [56] V. Nair and G. E. Hinton. Rectified linear units improve restricted boltzmann machines. In *Icml*, 2010.
- [57] A. Nigam, P. Sahare, and K. Pandya. Intent detection and slots prompt in a closed-domain chatbot. *arXiv preprint arXiv:1812.10628*, 2018.
- [58] J. Pennington, R. Socher, and C. D. Manning. Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 1532–1543, 2014.
- [59] L. Qin, W. Che, Y. Li, M. Ni, and T. Liu. Dcr-net: A deep co-interactive relation network for joint dialog act recognition and sentiment classification. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 8665–8672, 2020.
- [60] V. Raheja and J. Tetreault. Dialogue act classification with context-aware self-attention. *arXiv preprint arXiv:1904.02594*, 2019.
- [61] R. R. d. Resende de Mendonça, D. F. d. Felix de Brito, F. d. F. de Franco Rosa, J. C. dos Reis, and R. Bonacin. A framework for detecting intentions of criminal acts in social media: A case study on twitter. *Information*, 11(3):154, 2020.
- [62] I. Rish et al. An empirical study of the naive bayes classifier. In *IJCAI 2001 workshop on empirical methods in artificial intelligence*, volume 3, pages 41–46, 2001.

- [63] J. Schuurmans and F. Frasincar. Intent classification for dialogue utterances. *IEEE Intelligent Systems*, 35(1):82–88, 2019.
- [64] S. Schwarz, A. Theóphilo, and A. Rocha. Emet: Embeddings from multilingual-encoder transformer for fake news detection. In *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 2777–2781. IEEE, 2020.
- [65] J. R. Searle. A classification of illocutionary acts. *Language in society*, 5(1):1–23, 1976.
- [66] B. Settles. Active learning. *Synthesis lectures on artificial intelligence and machine learning*, 6(1):1–114, 2012.
- [67] K. Shridhar, A. Sahu, A. Dash, P. Alonso, G. Pihlgren, V. Pondeknath, F. Simistira, and M. Liwicki. Subword semantic hashing for intent classification on small datasets. *arXiv preprint arXiv:1810.07150*, 2018.
- [68] G. Singh, J. Thomas, and J. Shawe-Taylor. Improving active learning in systematic reviews. *arXiv preprint arXiv:1801.09496*, 2018.
- [69] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1):1929–1958, 2014.
- [70] S. Subramani, H. Q. Vu, and H. Wang. Intent classification using feature sets for domestic violence discourse on social media. In *2017 4th Asia-Pacific World Congress on Computer Science and Engineering (APWC on CSE)*, pages 129–136. IEEE, 2017.
- [71] S. Sukhbaatar and R. Fergus. Learning from noisy labels with deep neural networks. *arXiv preprint arXiv:1406.2080*, 2(3):4, 2014.
- [72] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. Going Deeper with Convolutions. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 1–9, 2015.
- [73] C. Tan, F. Sun, T. Kong, W. Zhang, C. Yang, and C. Liu. A survey on deep transfer learning. In *International Conference on Artificial Neural Networks*, pages 270–279. Springer, 2018.
- [74] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, pages 5998–6008, 2017.
- [75] Y. Wang, L. Tang, and T. He. Attention-based cnn-blstm networks for joint intent detection and slot filling. In *Chinese Computational Linguistics and Natural Language Processing Based on Naturally Annotated Big Data*, pages 250–261. Springer, 2018.

- [76] C. Welch, A. Lahnala, V. Pérez-Rosas, S. Shen, S. Seraj, L. An, K. Resnicow, J. Pennebaker, and R. Mihalcea. Expressive interviewing: A conversational system for coping with covid-19. *arXiv preprint arXiv:2007.03819*, 2020.
- [77] C.-S. Wu, S. Hoi, R. Socher, and C. Xiong. Tod-bert: Pre-trained natural language understanding for task-oriented dialogues. *arXiv preprint arXiv:2004.06871*, 2020.
- [78] Z. Yang, Z. Dai, Y. Yang, J. Carbonell, R. Salakhutdinov, and Q. V. Le. Xlnet: Generalized autoregressive pretraining for language understanding. *arXiv preprint arXiv:1906.08237*, 2019.
- [79] L. Zhang and L. Zhang. An ensemble deep active learning method for intent classification. In *Proceedings of the 2019 3rd International Conference on Computer Science and Artificial Intelligence*, pages 107–111, 2019.